NO SAFE HAVEN
PL programación avanzada



Grado en ingeniería informática
Curso 2024/2025 – Convocatoria Ordinaria

09109126E- Martínez Toledo, Cristina
03248484X – Fernández Ambrós, Alejandro

ÍNDICE

ÍNDICE **2**
1. Introducción **3**
3. Estructura del proyecto **3**
3.1 Clases principales 4
3.1.1 Zombie 4
3.1.2 Humano 5
3.1.3 Refugio 6
3.1.4 Tunel 7
3.1.5 ZonaExterior 8
3.1.6 Ranking 9
3.1.7 EstadoPausa 9
3.1.8 Velocidad 10
3.1.9 Log 10
3.1.10 Cliente 10
3.1.11 Servidor 11
3.2 Clases main 12
3.2.1 MainCliente 12
3.2.2 NoSafeHaven (main del servidor) 12
3.3 Interfaz gráfica 13
3.3.1 JDialogCliente 13
3.3.2 JDialogServidor 13
3.3.3 ClientePantalla 14
3.3.4 PantallaPrincipal 15
4. Mejoras añadidas **16**
5. Posibles Deadlocks **17**
6. Posibles mejoras **18**
7. Anexos **19**
7.1. Código 19

1. Introducción

En este trabajo, se ha desarrollado un proyecto de simulación titulado "No Safe Haven", cuyo objetivo principal es representar un escenario de apocalipsis zombie mediante técnicas de programación concurrente. Este entorno simulado incluye humanos que buscan sobrevivir desplazándose entre zonas de riesgo, túneles y un refugio, mientras evitan el ataque constante de zombis. El proyecto busca aplicar y consolidar conocimientos sobre concurrencia, sincronización entre hilos, estructuras de datos compartidas y diseño de interfaces gráficas en Java.

2. Diseño general del sistema y discusión de las herramientas de sincronización utilizadas

En “No Safe Haven” se implementa una simulación concurrente en Java donde cada humano y cada zombi se ejecutan en su propio hilo, poniendo a prueba mecanismos de sincronización y seguridad. Primeramente la clase Refugio maneja los suministros de comida con un ReentrantLock y todas las variables de datos como el número de humanos por zona haciendo uso de variables atómicas, al ser contadores simples, y listas de CopyOnWriteArrayList. La clase Tunel controla el paso de hilos humanos mediante un ReentrantLock, dos Conditions y una CyclicBarrier de tres participantes, para garantizar que solo uno cruce a la vez y que los que entran tengan prioridad, evitando condiciones de carrera. Cuando un Zombie selecciona y ataca a un Humano en una ZonaExterior, ambos hilos deben coordinarse mediante wait/notify para suspender y reanudar correctamente la simulación de la pelea y actualizar los contadores atómicos, las listas de CopyOnWriteArrayList y los registros en la clase Log, cuya escritura sincronizada evita pérdidas de información. Para mantener un podio de los zombis más letales, la clase Ranking emplea un método synchronized que ordena y recorta la lista de “bodycounts”, asegurando exclusión mutua. El componente EstadoPausa utiliza un Lock y una Condition para detener o reanudar todos los hilos en puntos clave, mientras que Velocidad actúa como factor global en los sleeps de los hilos para ajustar el ritmo de la simulación. Finalmente, las clases Servidor y Cliente ofrecen una interfaz cliente-servidor que recibe y envía comandos a través de una conexión TCP de pausa, velocidad o provisión de recursos alimenticios, integrando todas las acciones concurrentes con la actualización de la GUI. Este conjunto de actores principales —Humano, Zombie, Tunel, ZonaExterior, Refugio, Ranking, EstadoPausa, Velocidad, Log, Cliente y Servidor— conforma un entorno de estudio práctico sobre sincronización, bloqueo y coordinación de hilos en Java.

3. Estructura del proyecto

El proyecto se estructura en once clases principales y dos clases “main” en el backend, mientras que la interfaz gráfica se compone de cuatro pantallas, dos pantallas principales y dos pantallas de conexión o de “inicio”.

3.1 Clases principales

A continuación, se explicará la estructura, atributos y métodos principales de cada una de las once clases creadas, así como los métodos de sincronización utilizados en cada caso.

3.1.1 Zombie

La clase Zombie representa a los personajes no humanos del sistema que surgen cuando un humano muere, además del “paciente 0” o primer zombie creado al iniciarse el sistema. Zombie hereda de Thread, permitiendo que cada zombie se ejecute de forma concurrente dentro de la simulación. Cada zombi tiene un identificador id, un contador de víctimas bodycount, una referencia a las cuatro zonas exteriores (ZonaExterior[] zonas), una instancia del sistema de puntuación o clasificación Ranking, y una referencia al estado de pausa global (EstadoPausa), que permite sincronizar la ejecución con pausas controladas por el usuario o la lógica del sistema. El constructor de Zombie recibe el id del humano original, las zonas exteriores, el ranking y el estado de pausa. El id se transforma al formato de zombi sustituyendo la letra inicial por "Z" ("Z" + id.substring(1)), asegurando que el nuevo zombi herede la identidad del humano pero con el prefijo correcto.

En su método run(), el zombi entra en un bucle infinito de comportamiento que persiste mientras la simulación está activa. Este bucle contiene la lógica principal de patrullaje, ataque e interacción con humanos. El zombi escoge aleatoriamente una de las cuatro zonas exteriores (del 0 al 3) y se desplaza a ella utilizando el método “zombieLlegar” de la zona correspondiente.

Una vez en la zona seleccionada, ejecuta la función “zombieAtacar”, en la que si hay algún humano en la zona, el zombie le atacará. De todos los humanos en la zona, se seleccionará uno aleatoriamente, se le sacará de la lista de humanos en esa zona para evitar que otro zombie le ataque, y se simulará la pelea. El humano tendrá ⅔ de probabilidades de sobrevivir, estado “calculado” en la función zombieAtacar. Al calcular el resultado, se actualiza inmediatamente el estado del humano. Después de calcularlos, se llamará a la función “atacar” del zombie y se interrumpirá al humano con setMuerto o setHerido. Después de calcularlos, se llamará a la función “atacar” del zombie y se interrumpirá al humano para que, como se explicará posteriormente, espere a que el ataque finalice. En la función “atacar” el zombie dormirá con sleep un tiempo aleatorio entre 0.5 y 1.5 segundos, tras lo que escribirá el resultado del ataque por pantalla y en el archivo de log. Además, si ha matado al humano se aumentará su contador de muertes y se llamará a la función de actualizar el ranking de la clase Ranking (por si este tuviera que actualizarse).

Finalmente, el zombie sale de la zona con “zombieIrse” y se dirige a otra, tras haber esperado durante un período aleatorio de 2 a 4 segundos, simulando el tiempo que tarda en moverse de zona o el tiempo que ha permanecido en esa área antes de moverse a otra.

El sistema de pausa (estadoPausa.parar()) se invoca de forma recurrente a lo largo del método run() para garantizar que la simulación pueda detenerse en cualquier momento de forma

segura. También, escribe por pantalla y registra en el log todas las acciones que va realizando en cada iteración.

3.1.2 Humano

La clase Humano representa a un individuo dentro de la simulación. Hereda de Thread, lo que permite que cada instancia se ejecute de forma concurrente en su propio hilo. Su objetivo principal es simular el comportamiento de un humano que se desplaza entre el refugio y las zonas exteriores buscando comida, descansando, comiendo, siendo atacado y eventualmente, si muere, convirtiéndose en un zombi.

El código incluye varios métodos que serán llamados desde el método run y encapsulan acciones del humano, como “irZonaComun()”, “salirRefugio(int tunel)”, “irComedor()”, etc, creados para simplificar el propio método run y hacerlo más legible.

Al comenzar su ejecución con el método run, el humano se une al refugio, incrementando el contador global de humanos. A partir de ese momento, entra en un ciclo que se repite mientras el humano no esté marcado como muerto (momento en el cual se convertirá en zombie). En cada iteración de este ciclo realiza varias acciones:
Primero, visita la zona común del refugio y elige aleatoriamente un túnel para salir al exterior. Entonces, por ese túnel saldrá a una de las cuatro zonas exteriores; la correspondiente al túnel seleccionado, y el hilo entrará en un “sleep” de tres a cinco segundos para simular que se encuentra buscando comida. Si en este momento un zombie llega a la zona y selecciona al humano en cuestión, se interrumpirá el sleep con un interrupt. Si es interrumpido, se simulará la pelea. Si el humano sobrevive, será herido y volverá sin comida al túnel, mientras que si muere deja de ejecutarse como Humano y se crea un nuevo hilo de tipo Zombie, representando su transformación. Para simular esta pelea, mientras que el zombie ejecuta la función “atacar” vista anteriormente, entrará en un “wait” gracias a un método de la propia zona exterior. Gracias a esto, tanto el humano como el zombie “esperarán” a la vez el mismo tiempo de lucha, y el humano no “escapará” hasta que se decida el resultado de la pelea.

Si sobrevive, vuelve por el mismo túnel al refugio, donde si no ha sido herido, deja la comida recolectada. Entonces, irá a descansar (2-4 segundos) y alimentarse en el comedor, donde esperará si no hay comida. Si ha sido herido, volverá a descansar de nuevo para curarse, ahora 3-5 segundos. Finalmente, si el humano sigue vivo, volverá a repetirse el mismo ciclo

La clase interactúa con múltiples recursos compartidos entre hilos:

- *Refugio*: entre otras cosas, como se verá proximamente, regula el acceso a las zonas comunes, comedor y túneles
- *ZonaExterior[] zonas*: es un arreglo de zonas de riesgo a las que los humanos acceden para buscar comida.

- *EstadoPausa*: es una clase que se invoca con frecuencia mediante estadoPausa.parar(), para comprobar si se debe pausar/resumir la simulación en ese momento exacto.
- *Velocidad.getVelocidad()*: para ajustar el ritmo de la simulación

Además, el Humano escribe mensajes por consola (System.out.println) de todas sus acciones, las registra en el archivo de log para la ejecución, usando la clase Log, lo que permite observar el comportamiento en tiempo real y hacer un seguimiento de eventos importantes.

3.1.3 Refugio

La clase “Refugio” representa el núcleo organizativo y de control dentro del sistema simulado del apocalipsis. Su principal función es coordinar y sincronizar las actividades de los humanos que entran, se alimentan, descansan o se trasladan por túneles, así como gestionar la comida disponible y mantener actualizadas las métricas visibles en la interfaz gráfica y el servidor.

La clase mantiene una serie de contadores atómicos (“AtomicInteger”) que registran el número actual de humanos en el refugio, el total de humanos y zombis en la simulación, así como los recursos alimenticios disponibles. Se utilizan estructuras concurrentes como “CopyOnWriteArrayList” para representar las listas de humanos presentes en la zona común, zona de descanso y comedor, asegurando acceso seguro y eficiente desde múltiples hilos, además de permitir que las operaciones de lectura se realicen sin necesidad de bloquear la lista. Junto a esto, “Lock y Condition” son usados para manejar la sincronización cuando los humanos esperan comida en lugar de synchronized para evitar bloquear el refugio entero. Además, Refugio contiene un array de objetos “Tunel”, cada uno representando un acceso al exterior. En el constructor, se inicializan los cuatro túneles y se recibe una instancia del Servidor, responsable de actualizar los datos visuales y de control en tiempo real en la interfaz del cliente.

Como ampliación, en la interfaz del cliente se mostrará un conteo del total de humanos y zombies en el sistema. Los métodos unHumanoMas y unHumanoMenos incrementan y decrementan el número total de humanos respectivamente, reflejando estos cambios en consola, logs y la interfaz con llamadas al servidor. De forma similar, unZombieMas y unZombieMenos mantienen el conteo de zombies activos en la simulación.

Cuando un humano entra al refugio, llamará a humanoEntraRefugio para aumentar el valor total de humanos en el refugio y actualizar las interfaces, incluyendo una llamada al servidor para que se actualice la interfaz del cliente. De forma similar, con humanoSaleRefugio se refleja que el humano ha salido por el túnel del refugio.

También hay dos métodos para incrementar la comida: addComida(int comidaExtra), como mejora añadida, para añadir una cantidad específica desde el cliente, y dejarComida(), usada por los humanos, que incrementa por defecto en dos unidades. Ambos métodos notifican a los hilos en espera mediante esperarComida.signalAll(), además de actualizar las interfaces

incluyendo una llamada al servidor para que también se actualice la del cliente. Con el método irComedor(String id), un humano que llega al comedor verifica si hay comida disponible; si no la hay, queda bloqueado esperando hasta que otro hilo lo despierte tras reponer las reservas. Cuando puede comer, el contador de comida se decrementa y de nuevo se actualiza la interfaz y el servidor. Al terminar, el método salirComedor elimina al humano del comedor y actualiza la visualización de los humanos en esa zona.

Para los desplazamientos de humanos dentro del refugio, la clase define métodos para entrar y salir de la zona común, de la zona de descanso, y del comedor. Estas acciones modifican tanto las listas internas como los contadores asociados, y notifican los cambios a la interfaz.

En cuanto a los túneles, la clase proporciona los métodos salirRefugio, entrarRefugio y salirTunel, para realizar las llamadas a los métodos correspondientes del túnel, siempre que el túnel al que se intenta acceder exista (es decir, sea un túnel con id entre 0 y 3 en este caso). En salirRefugio, para la transición desde la zona común hasta fuera del refugio, además se reduce el contador de humanos y se llama a humanoSaleRefugio.

3.1.4 Tunel

La clase Túnel representa un recurso compartido por el que los humanos entran o salen del refugio en la simulación. El túnel permite que solo un humano cruce a la vez. Para garantizar esto, se utiliza un ReentrantLock (entrando) junto con dos Condition (quiereEntrar y quiereSalir) que representan las colas de espera de los humanos que quieren entrar y los que quieren salir del refugio, respectivamente, además de una variable que indica cuántos humanos hay dentro del túnel (1 o 0).

El comportamiento de entrada y salida se gestiona para evitar colisiones entre humanos que entran y salen al mismo tiempo. Los humanos que quieren entrar al refugio tienen prioridad. Para establecer esta prioridad, se utiliza una variable que cuenta cuántos humanos quieren entrar. De esta forma, los humanos que quieran entrar deberán esperar únicamente a que el túnel esté libre, mientras que un humano que quiere salir del refugio tiene que esperar a que el túnel esté libre y además que no haya nadie que quiera entrar. Además, para respetar esa prioridad, cuando un humano sale del túnel con salirTunel, si hay humanos esperando a entrar en la Condition quiereEntrar, despertará a los de ese grupo, y si no hay, despertará a los que esperan en quiereSalir. En el método salirRefugio se utiliza una CyclicBarrier, que obliga a que tres humanos estén listos para salir por ese túnel antes de proceder. Una vez los tres llegan, todos avanzan, y luego se bloquean según las reglas de prioridad definidas por los Lock y Condition ya indicadas.

La clase utiliza tres listas CopyOnWriteArrayList para guardar los IDs de los humanos en cada zona del túnel, y se utilizarán además para actualizar la interfaz gráfica (PantallaPrincipal) en tiempo real:

- -humanosSaliendoRef: humanos que están esperando para salir del refugio.
- -humanosEntrandoRef: humanos que están intentando entrar al refugio.
- -humanosDentroRef: humanos que están actualmente dentro del túnel.

Cada acción que cambia el estado del túnel llama a actualizarHumanosEnTunel(), que informa al Servidor del número actual de personas implicadas combinando el tamaño de las tres listas. Además, con cada acción se actualiza visualmente cada sección (fuera del túnel, dentro, saliendo) con llamadas al método correspondiente de la pantalla principal.

3.1.5 ZonaExterior

La clase ZonaExterior representa una de las cuatro áreas exteriores del refugio en la simulación de apocalipsis zombie. En este entorno, tanto humanos como zombies pueden entrar y salir, interactuar mediante ataques y modificar el estado global del sistema. Cada ZonaExterior se identifica mediante un ID, mantiene una referencia al Servidor y se comunica con interfaz gráfica (PantallaPrincipal). Se utilizan contadores atómicos (AtomicInteger) para almacenar el número de humanos (ctdhumanos) y zombies (ctdzombies) en la zona, así como listas seguras tipo CopyOnWriteArrayList para almacenar instancias de humanos y zombis activos en la zona, junto con listas paralelas que mantienen sus respectivos IDs para visualización.

Los zombis interactúan con la zona exterior mediante tres métodos principales:

- *zombieLlegar(Zombie z)*: añade al zombi a las listas internas, incrementa el contador de zombis y actualiza tanto la interfaz gráfica como el servidor.
- *zombieAtacar(Zombie z)*: si hay humanos disponibles, selecciona uno aleatoriamente y lo ataca, modificando antes y después del ataque la variable isSiendoAtacado del humano. Hay una probabilidad del 33% de que el humano muera. En cualquier caso, el humano atacado se elimina de la zona (para que no sea atacado de nuevo) y se interrumpe su hilo. Finalmente, se llama a z.atacar(...) para registrar el evento y se notifica al hilo en espe*ra del humano de que el ataque ha terminado.*
- zombieIrse(Zombie z): remueve al zombi de las listas, decrementa el contador y actualiza los datos visuales y del servidor.

Los humanos también interactúan con la zona a través de métodos simétricos:

- *humanoLlegar(Humano h)*: añade al humano a las listas, incrementa el contador, y actualiza la interfaz gráfica y el servidor.
- *humanoIrse(Humano h)*: elimina al humano de las listas, decrementa el contador, y actualiza estado e interfaz. Se llama desde zombieAtacar() después de un ataque, independientemente del resultado.
- *humanoAtacado(Humano h)*: se utiliza para pausar al humano si está siendo atacado. Usa un bucle while (isSiendoAtacado) con wait() para bloquear su ejecución hasta que se le notifique el final del ataque.

Todas las interacciones (llegadas, salidas, ataques) resultan en llamadas a los métodos correspondientes de la pantalla principal para actualizar las vistas en tiempo real, salidas por pantalla y escrituras en el log, además de llamadas al servidor cuando se modifican los contadores totales para actualizar los datos de la interfaz del cliente.

3.1.6 Ranking

La clase Ranking se encarga de mantener actualizado el podio de los tres zombis más letales del sistema en función de su bodycount (número de humanos eliminados)
La clase mantiene una lista privada ranking, que almacena strings en el formato "Id del zombie: número de muertes". También tiene una referencia al objeto Servidor para hacer llamadas y actualizar los datos visibles en la interfaz del cliente.

El método principal actualizarRanking está marcado como synchronized, lo que garantiza exclusión mutua cuando múltiples zombis intentan actualizar el ranking al mismo tiempo. Esto es esencial en un entorno multihilo para evitar condiciones de carrera.

La lógica sigue estos pasos:

- Eliminación previa: Si el ID del zombi ya estaba en la lista (es decir, ya tenía un ranking anterior), se elimina esa entrada.
- Adición del nuevo dato: Se agrega una nueva entrada con el ID actualizado y su bodycount.
- Ordenación: La lista se ordena de mayor a menor según el número de muertes. Para ello se usa el método auxiliar extraerValor, que obtiene el valor numérico a partir del formato "Z123: 5".
- Recorte del ranking: Se limita la lista a los tres primeros elementos
- Notificación al servidor: Se llama a servidor.actualizarRanking(ranking)
- Visualización en consola: Se imprime el ranking actualizado

3.1.7 EstadoPausa

La clase EstadoPausa se utiliza para el control de la pausa y reanudación de la simulación. Esta clase utiliza un ReentrantLock (pauseLock) para garantizar exclusión mutua cuando se cambia el estado de pausa o cuando un hilo consulta si debe esperar, una Condition (pauseCondition) asociada al lock, que permite suspender y reanudar la ejecución de múltiples hilos, y un booleano paused declarado como volatile para garantizar la visibilidad entre hilos, permitiendo que todos los hilos lean correctamente su valor actualizado incluso sin necesidad de adquirir el lock.

Todos los hilos (humanos y zombis) llaman al método parar() en puntos clave de su ejecución (antes y después de dormir, antes de entrar/salir de zonas, etc.). Si el usuario activa la pausa

desde la interfaz, todos los hilos se suspenden bloqueandose en la condition al llegar a su siguiente llamada a parar(), y al reanudarla, se ejecuta un notifyAll y todos los hilos continúan automáticamente.

3.1.8 Velocidad

Esta clase, creada como mejora, se utiliza para modificar la velocidad de las acciones del programa. Al dividir todos los tiempos de sleep entre la variable estática que contiene, como si fuera una variable global, logramos dar el efecto de más velocidad. Desde la interfaz gráfica del cliente se puede modificar el valor de esta variable para acelerar la simulación al presionar uno de los tres botones.

3.1.9 Log

La clase Log tiene como propósito registrar eventos del sistema en un archivo de texto para facilitar la monitorización o depuración. Al iniciarse, genera automáticamente un nombre único para el archivo de log combinando una cadena base (apocalipsis) con la fecha y hora actual, formateada para ser válida en nombres de archivo. Este archivo se guarda dentro de una carpeta específica dentro del proyecto llamada logs. Contiene un método estático “escribir”, que permite añadir mensajes al archivo, cada uno precedido por una marca de tiempo (timestamp) con precisión de segundos. Este método está declarado como synchronized, lo que garantiza que únicamente un hilo puede ejecutarlo a la vez, protegiendo así la escritura concurrente y evitando condiciones de carrera. El uso de try-with-resources con PrintWriter para la escritura asegura que el archivo se cierre correctamente tras cada escritura, incluso si se produce una excepción, evitando así posibles pérdidas de datos.

3.1.10 Cliente

La clase Cliente representa el componente cliente de una arquitectura cliente-servidor en la simulación de No Safe Haven. Su propósito principal es establecer una conexión con el servidor, recibir mensajes de datos en tiempo real y actualizar la interfaz gráfica remota (ClientePantalla) según los cambios detectados.

Primero, con el método conectarAlServidor(String host, int puerto), se crea un Socket con el servidor en la dirección y puerto especificados. Una vez conectado, se inicializan los streams de entrada y salida, entrada con BufferedReader y salida con PrintWriter, y se cambia el estado de conexión a true, utilizado en el main del cliente según como se indicará posteriormente.

El método iniciarRecepcion lanza un hilo (receptor) encargado de escuchar continuamente la entrada de información del servidor. Este hilo detecta si el servidor envió el comando "SALIR" para finalizar correctamente la conexión y finalizar la ejecución, y procesa líneas que comienzan con "data", que contienen pares clave-valor separados por =. Según la clave

recibida, llama a métodos específicos de ClientePantalla para actualizar la visualización de los datos.

El método enviarComando(String comando) permite enviar órdenes simples al servidor, como "SALIR" y otros comandos registrados, mediante el PrintWriter. Se utiliza tanto en la desconexión como para enviar acciones interactivas desde la interfaz.

Finalmente, el método desconectar asegura una desconexión limpia y controlada, protegiéndose ante posibles estados nulos del socket, además de enviar el comando “SALIR” al servidor para finalizar también su ejecución. Este método se invocará al cerrar la interfaz.

3.1.11 Servidor

La clase Servidor representa el lado servidor de la arquitectura cliente-servidor en el sistema de la simulación. Su principal propósito es escuchar conexiones entrantes desde un cliente, recibir comandos de control en tiempo real y enviar información en tiempo real sobre el estado de la simulación (refugio, túneles, zonas de riesgo, ranking, etc.) a través de un socket.

En el método iniciarServidor, si está disponible el puerto elegido, se escucha en ese puerto específico mediante ServerSocket. Cuando un cliente se conecta, acepta la conexión (accept()), abre los streams de comunicación (entrada y salida) y cambia la variable conectado a true.

Si hubiese una conexión abierta en ese puerto, no se podría iniciar un nuevo servidor. Sin embargo si utilizáramos un try with resources donde inicializamos el ServerSocket del servidor, este se cerraría automáticamente al salir del bloque try habiendo creado la conexión con el cliente, lo que permitiría que se crearan distintos sockets o canales de comunicación entre un cliente y un servidor en el mismo puerto.

A continuación, lanza un hilo “comandos” que se ejecuta en paralelo para leer continuamente órdenes enviadas por el cliente. El hilo comandos interpreta los siguientes mensajes:

- -PAUSAR / REANUDAR: invoca métodos de EstadoPausa para congelar o liberar la ejecución de todos los hilos en la simulación.
- -VELOCIDAD=n: cambia la velocidad global de ejecución (afectando a sleep() de humanos y zombis).
- -COMIDA_EXTRA=n: inyecta comida extra directamente al refugio usando refugio.addComida.
- -SALIR: cierra el socket, termina la sesión y detiene el sistema con System.exit(0).

Todos los comandos son impresos en consola para trazabilidad

El servidor actúa también como emisor de actualizaciones en tiempo real utilizando el prefijo "data" y seguido de claves específicas esperadas por el cliente, con el formato clave=valor, en

los métodos “actualizarDatos…”. También se envían comandos con enviarComando (utilizado para enviar “SALIR” en el método desconectar)

Por último, con el método desconectar, el servidor envía el comando "SALIR" al cliente, cierra sus propios recursos (entrada, salida, clienteSocket) y muestra un mensaje por consola. Este método se invocará al cerrar la interfaz.

3.2 Clases main

3.2.1 MainCliente

Este método main es el punto de entrada del cliente en la simulación. Primero, se crea un objeto Cliente, que se encargará de conectarse al servidor, y se pasa la referencia del cliente a la interfaz gráfica ClientePantalla para que pueda, por ejemplo, cerrar la conexión desde la GUI.
Entonces, se mostrará una pequeña ventana de inicio o conexión (JDialogCliente), desde donde se introducirán los datos. Se permite un máximo de 3 intentos para conectarse al servidor; si se realizan tres conexiones fallidas, el programa finalizará. Además, el usuario puede decidir cerrar la ventana de conexión y cancelar el proceso (cerrar = true), finalizando también el programa. Si por el contrario se consigue establecer conexión, se cerrará la ventana de conexión, se lanzará el hilo de recepción de datos del servidor mediante el método iniciarConexión y se mostrará la interfaz principal del programa (ClientePantalla) , donde se visualizarán todos los datos recibidos.

3.2.2 NoSafeHaven (main del servidor)

La clase NoSafeHaven representa el punto de entrada principal del lado servidor de la simulación multihilo de un apocalipsis zombi.
La fase de conexión de esta clase funcionará de forma muy similar a la vista en la clase MainCliente. En primer lugar, se crea una instancia del objeto Servidor, responsable de comunicarse con el cliente, y se asocia con la interfaz PantallaPrincipal, que mostrará los datos de la simulación en tiempo real. A continuación se entra en un bucle que permite hasta tres intentos para conectarse, a través de un cuadro de diálogo (JDialogServidor) donde el usuario introduce el puerto. Si el usuario cierra el diálogo sin confirmar, la simulación no continúa.

Una vez establecida la conexión con éxito, se muestra la interfaz principal del servidor (PantallaPrincipal) y se crea una instancia de la clase EstadoPausa y Refugio, que se asocian al servidor para pausar la simulación y añadir comida al refugio al recibir los respectivos comandos del cliente.
Después se crean cuatro instancias de ZonaExterior y un objeto Ranking. Seguidamente, se crea el primer zombi del sistema, el "paciente cero", con ID "Z00000". Este zombi inicia su hilo de ejecución inmediatamente y se registra en el log. A continuación, se crean múltiples hilos de

humanos. En el ejemplo, se lanzan 10000 instancias de la clase Humano, cada una con un ID único con el formato HXXXXX. Entre la creación de un humano y otro se introduce una pequeña pausa aleatoria, ajustada con el valor de Velocidad, para evitar que todos los hilos se ejecuten simultáneamente desde el inicio. En cada iteración del bucle, hay una llamada a parar() de la instancia de EstadoPausa para detener la creación de humanos si fuese necesario (si se hubiera pedido desde el cliente).

3.3 Interfaz gráfica

3.3.1 JDialogCliente

Esta clase define una pequeña ventana modal (diálogo emergente) para que el cliente introduzca la dirección IP y el puerto del servidor al que desea conectarse. Cuenta con campos de entrada (JTextField) para estos datos y un botón para confirmar la conexión. Cuando el usuario hace clic en "COMENZAR", se validan los valores introducidos: la dirección no debe estar vacía y el puerto debe estar en el rango válido (1024–65535). Si los datos son válidos, se almacenan en variables internas (direccion, puerto) y el diálogo se cierra marcando confirmado = true, mientras que si no lo son, se mostrará un mensaje emergente con JOptionPane. Si por el contrario se cierra el diálogo con la "X", se marca la variable quiereSalir = true para que el main sepa que el usuario no desea continuar. Esta ventana será la mostrada al comenzar la ejecución del main del cliente, en el bucle de conexión, explicado en el punto 2.2.1. El diseño gráfico de la pantalla se muestra en la siguiente imagen:



3.3.2 JDialogServidor

Esta clase es una pequeña ventana modal que permite al usuario introducir el puerto en el que desea iniciar el servidor. Funciona de forma casi idéntica a JDialogCliente, pero enfocado solo en la configuración del servidor. Esta ventana se usa al principio del main del servidor, en el bucle de conexión ya explicado en el punto 2.2.2, para obtener el puerto antes de lanzar la simulación. La interfaz se ve de la siguiente forma:



3.3.3 ClientePantalla

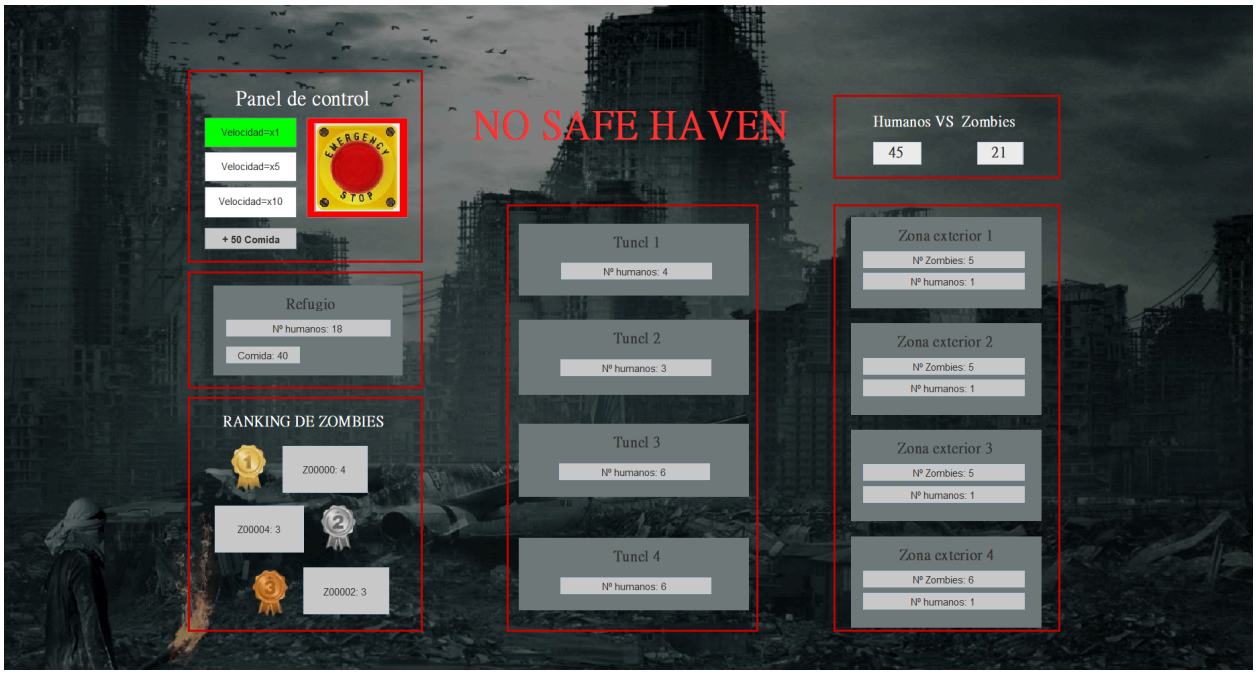
La clase ClientePantalla implementa una interfaz gráfica Swing para el cliente de una simulación multihilo, y sigue el patrón Singleton, lo que significa que solo puede haber una única instancia de esta clase durante toda la ejecución del programa. Esto se logra mediante un constructor privado y un método getInstancia() que emplea la técnica de "doble verificación con sincronización" para garantizar que la instancia se cree una unica vez en la primera llamada al método incluso si múltiples hilos la solicitan al mismo tiempo. La variable instancia está marcada como volatile, lo que asegura que los cambios realizados en ella por un hilo sean visibles inmediatamente para otros hilos. Esto es crucial en contextos multihilo como en esta simulación para evitar errores de visibilidad, donde un hilo podría estar viendo una versión obsoleta de la variable.

Toda la actualización de los componentes visuales (como los campos de texto que muestran número de humanos o zombis en diferentes zonas) se realiza mediante SwingUtilities.invokeLater(). Esto es necesario porque Swing no es thread-safe, es decir, no permite actualizar sus componentes desde hilos fuera del Event Dispatch Thread (EDT), que es el hilo que gestiona la GUI. Usar invokeLater() garantiza que los cambios visuales se realicen correctamente y sin errores de concurrencia, ya que que cuando la simulación (que corre en múltiples hilos) intenta actualizar visualmente un componente de la GUI, dicha actualización no se realiza directamente, sino que se "encola" para ejecutarse de forma segura.

Además de los métodos de actualización visual, la clase define acciones asociadas a botones. Estas acciones envían comandos al servidor a través de un objeto Cliente, que encapsula la lógica de comunicación por sockets. Uno de los botones es el de pausar y reanudar, que alterna el estado interno paused cada vez que se presiona. Cuando el usuario hace clic en él, el sistema envía al servidor un comando "PAUSAR" o "REANUDAR" a través del objeto cliente, indicando así que se debe detener temporalmente o continuar con la simulación. Además, el botón cambia de color entre verde y rojo, proporcionando una señal visual clara del estado actual. También destacan los botones de velocidad (x1, x5 y x10), que ajustan la rapidez con la que se ejecuta la simulación. Cada uno de ellos envía un comando específico al servidor (VELOCIDAD=x) y actualiza su color para mostrar cuál está activo. Finalmente, el botón "Comida Extra" simula una intervención externa para proporcionar recursos al refugio, enviando el comando "COMIDA_EXTRA=50" al servidor. Por último, la clase también intercepta el evento

de cierre de ventana (windowClosing) para ejecutar una desconexión ordenada del cliente, cerrando streams y el socket para evitar fugas de recursos, con una llamada a la función desconectar del cliente.

La interfaz se ve de la siguiente forma:



3.3.4 PantallaPrincipal

La clase PantallaPrincipal representa la interfaz gráfica del servidor de la simulación de apocalipsis zombi. Al igual que ClientePantalla, eEs una clase que hereda de JFrame y sigue el patrón Singleton para asegurar que solo exista una instancia visible durante la ejecución del programa, además de que la vista se actualiza de manera segura en el hilo de eventos de Swing con SwingUtilities.invokeLater().

Una de las características destacables es el uso de un Set<String>, colección que almacena elementos de tipo String sin permitir duplicados, llamado heridos, sincronizado con un ReentrantLock. Esto se utilizará para resaltar ciertos humanos en color rojo en los distintos paneles gráficos. Los cuadros de texto donde puede haber humanos heridos se crearon como JTextPane, mientras que los que no, son simples JTextArea.

En los métodos para la actualización de los JTextArea, primero se crea un StringBuilder llamado sb para concatenar los IDs de los hilos que están en esa zona. Se agregan con espacios y saltos de línea cada 3 elementos, para que se vean organizados. Después, dentro del invokeLater hay una simple llamada a setText

Por otra parte, para actualizar los JTextPane se utilizan varios métodos públicos, que contienen llamadas al método empleado para pintar los elementos (actualizarTextPaneConColor), que detecta si un ID está marcado como herido y aplica estilos personalizados (SimpleAttributeSet) a los cuadros de texto para cada ID. Esto permite diferenciar visualmente a los humanos heridos del resto, aumentando la legibilidad y seguimiento en tiempo real.
Este método primero prepara un nuevo StyledDocument, que es un tipo especial de documento para JTextPane que permite aplicar estilos diferentes a partes del texto, y configura dos estilos, estiloNormal (color negro) y estiloHerido (color rojo) con StyleConstants.setForeground(...). Después, se recorre cada ID de la lista verificando si el ID está en el conjunto idsHeridos; si lo está, lo escribe en el documento con estiloHerido, y si no, lo escribe con estiloNormal. Además, se controla el formato del texto: si el parámetro pasado porLinea == 1, se inserta un salto de línea por cada ID, y si porLinea > 1, los IDs se separan con espacios hasta llegar al numero especificado y despues se inserta salto de linea, para estructurar la presentación en varias líneas. Una vez completado el documento, se actualiza el JTextPane en el hilo de eventos utilizando nuevamente SwingUtilities.invokeLater, con textPane.setDocument().

El aspecto de la interfaz es el siguiente:



4. Mejoras añadidas

Durante el diseño y desarrollo de este proyecto, se identificaron varias mejoras que podían incorporarse sin interferir con los requisitos básicos. Estas mejoras aportan valor añadido, ya sea complementando determinados aspectos o facilitando la visualización de ciertas características. Las mejoras ya se han mencionado anteriormente, pero para facilitar la comprensión y lectura del documento se volverán a mencionar a continuación.

Una de las primeras mejoras implementadas fue la posibilidad de aumentar la velocidad de la simulación. Esto permite observar el comportamiento del sistema de forma más ágil y detectar posibles fallos con mayor rapidez. La velocidad puede ser modificada por el usuario mediante botones disponibles en la interfaz.

Otra mejora consistió en añadir un botón en la interfaz del cliente que, al ser pulsado, envía automáticamente 50 raciones de comida desde un refugio cercano. Esto sirve para evitar un deadlock cuando los humanos de nuestro refugio no pueden traer más comida y quedan bloqueados en el comedor, esperando a morir de inanición, como se explicará en el siguiente apartado.

Además, en lugar de crear un servidor y un cliente conectados siempre al mismo puerto, se creó una ventana modal, que además funciona como una pequeña pantalla de inicio, para permitir al usuario introducir los datos de la conexión.

Finalmente, se incorporó un contador de humanos vs zombis en la interfaz del cliente. Este indicador permite visualizar cuán cerca estamos del “fin de la humanidad”, además de ofrecer información adicional sobre el estado de la simulación.

5. Posibles Deadlocks

Debido a los requisitos del proyecto, se han detectado ciertos casos en los que se podría producir un interbloqueo.

Primeramente, debido a que se utiliza un CyclicBarrier para permitir la salida del refugio a los humanos de tres en tres, se suele dar el caso en los que esos hilos se queden esperando el paso infinitamente. Esto sucederá siempre que todos los hilos que queden accedan a túneles distintos, haciendo que en ninguno se forme un grupo completo para poder avanzar.

El segundo deadlock encontrado fue en el comedor, ya que si la comida es igual a cero los humanos se quedarán esperando a que otros humanos vuelvan y la “repongan”. Debido a esto, si no quedan más humanos para ir a buscar comida, los que ya estén esperando se quedarán esperando infinitamente. Además, incluso antes de que no queden más humanos por salir, en la ejecución llega un punto en que el número de zombies es tan elevado que es imposible para un humano salir del refugio y no morir o ser herido, y debido a que los humanos heridos no traen comida al refugio, el número de hilos bloqueados infinitamente seguirá aumentando, produciéndose de nuevo el mismo deadlock. Para paliar esto ligeramente, aunque de forma totalmente “manual” (mediante intervención del cliente), se creó como añadido la función de añadir comida, que añade 50 de comida al refugio, permitiendo por cada llamada a varios de los hilos salir de ese estado de bloqueo y continuar su ejecución.

6. Posibles mejoras

Una posible mejora para este proyecto en donde existe una alta concurrencia y gran cantidad de actualizaciones visuales es cambiar la forma en que se gestiona la actualización de la interfaz gráfica (GUI).

Cada vez que un hilo realiza una acción relevante (como moverse de una zona a otra, consumir comida, etc.) notifica inmediatamente a la interfaz gráfica para actualizar sus componentes. En escenarios con pocos hilos no sería ningún problema. Sin embargo, en este caso la simulación es bastante grande, teniendo hasta 10000 hilos en ejecución, por lo que es probable que cuando se realicen muchas llamadas a la interfaz con el EDT, y en muy poco tiempo, esta podría volverse lenta o intermitente.

Para aliviar este problema, una posible solución sería utilizar un ScheduledExecutorService que se encargue de actualizar la interfaz gráfica a intervalos regulares, por ejemplo, cada 100 o 200 milisegundos. En lugar de que cada hilo notifique la GUI de forma inmediata.

Otra mejora estructural a tener en cuenta consiste en reemplazar el uso explícito de locks (ReentrantLock) y condiciones (Condition) en ciertos métodos por estructuras de datos más adecuadas para ciertos escenarios, como colas.

Por ejemplo, en las zonas de comedor o los túneles, donde los recursos son limitados y los hilos deben esperar su turno para acceder, usar locks y condiciones para controlar el acceso puede llegar a introducir una complejidad considerable, como riesgo de deadlocks, manejo de prioridades, y necesidad de mantener múltiples condiciones.

En lugar de eso, es más directo y limpio utilizar una estructura como BlockingQueues (por ejemplo, ArrayBlockingQueue) para representar la cola de espera. Los hilos simplemente intentarían añadir su ID a la cola (put() o offer()), y si no hay espacio, esperarán automáticamente a que haya disponibilidad. Esto permite un acceso ordenado y seguro sin necesidad de gestionar explícitamente la sincronización. Además, si lo combinamos con una PriorityBlockingQueue también cubrimos la situación en la que los humanos fuera del refugio tienen más prioridad para entrar al túnel que los humanos que quieren salir.

Con respecto al servidor, se podría haber introducido un timeout al lanzar el servidor para que en caso de no recibir ninguna conexión en un tiempo determinado después del lanzamiento del programa, este finalizarse y así liberar el puerto ocupado para la conexión. Además, dependiendo de la interpretación, se podría modificar la forma de conexión entre el cliente y el servidor, permitiendo por ejemplo que el servidor comience sin esperar al cliente, que varios clientes se conecten al mismo servidor, o que después de que finalice el cliente, otro se pueda conectar.

7. Anexos

Junto a la memoria, se añaden los siguientes anexos:

- Proyecto completo de NetBeans
- Proyecto de Enterprise architect con los diagramas de clases correspondientes al backend y frontend, junto a imágenes de dichos diagramas, añadidos como un anexo debido a su gran extensión.
- Código escrito, en el siguiente apartado

7.1. Código

```
public class Humano extends Thread {
    private String id;
    private boolean herido = false;
    private boolean muerto = false;
    private boolean siendoAtacado = false;
    private Ranking r;
    private EstadoPausa estadoPausa;

    private Refugio refugio;
    private ZonaExterior[] zonas = new ZonaExterior[4];

    /**
     * Constructor de la clase Humano.
     *
     * @param id Identificador único del humano.
     * @param refugio Refugio asociado al humano.
     * @param zonas Array de zonas exteriores donde el humano puede interactuar.
     * @param r Ranking asociado al humano.
     * @param estadoPausa Estado de pausa actual del humano.
     */
    public Humano(String id, Refugio refugio, ZonaExterior[] zonas, Ranking r, EstadoPausa
estadoPausa) {
        this.id = id;
        this.refugio = refugio;
        this.zonas = zonas;
        this.r = r;
        this.estadoPausa = estadoPausa;
    }

    /**
     * Método que representa el ciclo de vida de un humano en el refugio durante
     * el apocalipsis. Este método se ejecuta en un hilo separado y controla las
     * acciones del humano, como buscar comida, interactuar con el refugio y
```

```
     * enfrentarse a posibles ataques de zombis.
     *
     * Comportamiento principal: - El humano entra al refugio y se registra en
     * el contador de humanos. - Se mueve entre diferentes zonas, como la zona
     * común, el exterior y las zonas de descanso. - Busca comida en el
     * exterior, enfrentándose a posibles ataques de zombis. - Si sobrevive,
     * regresa al refugio con o sin comida dependiendo de si fue herido. - Si
     * muere, se convierte en un zombi y se actualizan los contadores
     * correspondientes.
     *
     * Consideraciones: - El método utiliza pausas controladas por el estado de
     * pausa (`estadoPausa.parar()`). - Se maneja el tiempo de espera en el
     * exterior, que puede ser interrumpido por ataques. - Si el humano es
     * herido, no trae comida al refugio y necesita más tiempo de descanso. - Si
     * el humano muere, se crea un nuevo objeto `Zombie` y se inicia su hilo.
     *
     * Manejo de errores: - Se capturan excepciones generales para registrar
     * errores en el log y en la consola.
     */
    @Override
    public void run() {
        estadoPausa.parar();
        refugio.unHumanoMas(); //cuando llega un humano, el contador de humanos aumenta
        refugio.humanoEntraRefugio(id); //cuando llega un humano, el contador de humanos
aumenta
        while (!muerto) {
            try {

                irZonaComun();
                int tunel = (int) (Math.random() * 4); //elegir entre los tuneles 0-3 para salir del refugio
                Log.escribir(id + " sale de la zona comun.");
                System.out.println(id + " sale de la zona comun.");
                salirRefugio(tunel);
                estadoPausa.parar();

                zonas[tunel].humanoLlegar(this);
                Log.escribir(id + " busca comida en la zona exterior " + tunel + ".");
                System.out.println(id + " busca comida en la zona exterior " + tunel + ".");
                estadoPausa.parar();

                try {
                    sleep((int) ((Math.random() * 2000 + 3000) / Velocidad.getVelocidad())); // en
exterior 3-5 seg
                    estadoPausa.parar();
```

```
            } catch (InterruptedException e) {
                Log.escribir(id + " fue interrumpido mientras buscaba comida." + e.getMessage());
                System.out.println(id + " fue interrumpido mientras buscaba comida." +
e.getMessage());

                zonas[tunel].humanoAtacado(this);
                estadoPausa.parar();
            }

            zonas[tunel].humanoIrse(this);

            if (!muerto) {
                Log.escribir(id + " deja la zona exterior " + tunel + ".");
                System.out.println(id + " deja la zona exterior " + tunel + ".");

                estadoPausa.parar();
                entrarRefugio(tunel);
                if (!herido) {
                    refugio.dejarComida();
                    Log.escribir(id + " deja comida en el refugio.");
                    System.out.println(id + " deja comida en el refugio.");
                } else {
                    //si ha sido herido, trae comida
                    Log.escribir(id + " fue herido, no trae comida.");
                    System.out.println(id + " fue herido, no trae comida.");
                }

                irZonaDescansoGeneral(2000);
                irComedor();
                if (herido) {
                    irZonaDescansoHerido(3000);
                }
            } else {
                //crear zombie
                estadoPausa.parar();
                Log.escribir(id + " ha muerto y se convierte en ZOMBIE.");
                System.out.println(id + " ha muerto y se convierte en ZOMBIE.");
                Zombie z = new Zombie(id, zonas, r, estadoPausa);
                refugio.unHumanoMenos(); //cuando un humano muere, el contador de humanos
disminuye
                refugio.unZombieMas(); //cuando un humano muere, el contador de zombis
aumenta
                z.start();
                //acaba ejecucion. no vuelve a entrar al while."se convierte"
```

```
            }

        } catch (Exception e) {
            Log.escribir("Error en el hilo de " + id + ": " + e.getMessage());
            System.out.println("Error en el hilo de " + id + ": " + e.getMessage());
        }
    }
}

/**
 * Permite que un humano entre en una zona de descanso, permanezca allí
 * durante un tiempo determinado y luego salga de la zona de descanso.
 * Durante este proceso, se registran mensajes en el log y en la consola
 * para indicar las acciones realizadas.
 *
 * Internamenmte s emaneja una InterruptedException para manejar la
 * interrupción del hilo
 *
 * @param tiempomin El tiempo mínimo (en milisegundos) que el humano debe
 * permanecer en la zona de descanso. El tiempo real será un valor aleatorio
 * entre 2 y 4 segundos, ajustado por la velocidad del sistema.
 */
public void irZonaDescansoGeneral(int tiempomin) {
    estadoPausa.parar();
    refugio.irZonaDescanso(id);

    Log.escribir(id + " entra en la zona de descanso.");
    System.out.println(id + " entra en la zona de descanso.");

    estadoPausa.parar();
    try {
        sleep((int) ((Math.random() * 2000 + tiempomin) / Velocidad.getVelocidad())); //en zona
descanso 2-4 seg
    } catch (InterruptedException ex) {
        System.out.println(ex.getMessage());
    }
    estadoPausa.parar();
    refugio.salirZonaDescanso(id);
    Log.escribir(id + " sale de la zona de descanso.");
    System.out.println(id + " sale de la zona de descanso.");
    estadoPausa.parar();
}
```

```
    /**
     * Permite que un humano entre en la zona de descanso para curarse si está
     * herido. Durante este proceso, se registran mensajes en el log y en la
     * consola para indicar las acciones realizadas. El humano espera un tiempo
     * determinado antes de salir de la zona de descanso.
     *
     * Internamenmte se maneja una InterruptedException para manejar la
     * interrupción del hilo
     *
     * @param tiempomin El tiempo mínimo (en milisegundos) que el humano debe
     * permanecer en la zona de descanso. El tiempo real será un valor aleatorio
     * entre 2 y 4 segundos, ajustado por la velocidad del sistema.
     */
    public void irZonaDescansoHerido(int tiempomin) {
        estadoPausa.parar();
        refugio.irZonaDescanso(id);

        Log.escribir(id + " esta herido y entra en la zona de descanso para curarse.");
        System.out.println(id + " esta herido y entra en la zona de descanso para curarse.");

        estadoPausa.parar();
        try {
            sleep((int) ((Math.random() * 2000 + tiempomin) / Velocidad.getVelocidad())); //en zona
descanso 3-5 seg
        } catch (InterruptedException ex) {
            System.out.println(ex.getMessage());
        }
        estadoPausa.parar();
        herido = false; //se cura si esta herido
        PantallaPrincipal.getInstancia().quitarHerido(id);
        refugio.salirZonaDescanso(id);
        Log.escribir(id + " sale de la zona de descanso.");
        System.out.println(id + " sale de la zona de descanso.");
        estadoPausa.parar();
    }

    /**
     * Permite que un humano entre en el comedor, permanezca allí durante un
     * tiempo determinado y luego salga del comedor. Durante este proceso, se
     * registran mensajes en el log y en la consola para indicar las acciones
     * realizadas.
     *
     * Internamenmte se maneja una InterruptedException para manejar la
     * interrupción del hilo
```

```
     */
    public void irComedor() {
        estadoPausa.parar();
        refugio.irComedor(id);
        Log.escribir(id + " entra en el comedor.");
        System.out.println(id + " entra en el comedor.");
        estadoPausa.parar();
        try {
            sleep((int) ((Math.random() * 2000 + 3000) / Velocidad.getVelocidad())); //comiendo 3-5
seg
        } catch (InterruptedException ex) {
            System.out.println(ex.getMessage());
        }
        estadoPausa.parar();
        refugio.salirComedor(id);
        Log.escribir(id + " sale del comedor.");
        System.out.println(id + " sale del comedor.");
        estadoPausa.parar();
    }

    /**
     * Permite que un humano entre en el refugio a través de un túnel
     * específico. Durante este proceso, se registran mensajes en el log y en la
     * consola para indicar las acciones realizadas. El humano espera un tiempo
     * determinado antes de entrar al refugio.
     *
     * @param tunel El número del túnel por el cual el humano intenta entrar al
     * refugio.
     */
    public void entrarRefugio(int tunel) {
        estadoPausa.parar();
        Log.escribir(id + " intenta entrar al refugio por el tunel " + tunel + ".");
        System.out.println(id + " intenta entrar al refugio por el tunel " + tunel + ".");
        refugio.entrarRefugio(tunel, id); //... entra al tunel...
        estadoPausa.parar();
        try {
            sleep(1000 / Velocidad.getVelocidad()); //esperar 1 seg cruzar tunel
        } catch (InterruptedException ex) {
            System.out.println(ex.getMessage());
        }
        estadoPausa.parar();

        refugio.salirTunel(tunel, id); //...llega a dentro del refugio
        refugio.humanoEntraRefugio(id); // aumentar el contador de humanos en el refugio
```

```
        Log.escribir(id + " ha entrado al refugio por el tunel " + tunel + ".");
        System.out.println(id + " ha entrado al refugio por el tunel " + tunel + ".");
        estadoPausa.parar();
    }

    /**
     * Permite que un humano salga del refugio a través de un túnel específico.
     * Durante este proceso, se registran mensajes en el log y en la consola
     * para indicar las acciones realizadas. El humano espera un tiempo
     * determinado antes de salir completamente del refugio.
     *
     * @param tunel El número del túnel por el cual el humano intenta salir del
     * refugio.
     */
    public void salirRefugio(int tunel) {
        estadoPausa.parar();
        Log.escribir(id + " intenta salir del refugio por el tunel " + tunel + ".");
        System.out.println(id + " intenta salir del refugio por el tunel " + tunel + ".");

        refugio.salirRefugio(tunel, id); // "sale" de la zona comun y espera en la entrada del tunel
        //la llamada a refugio.humanoSalerefugio está dentro del propio metodo salirRefugio para
que se refleje mejor en la interfaz

        estadoPausa.parar();
        try {
            sleep(1000 / Velocidad.getVelocidad()); //esperar 1seg cruzar tunel
        } catch (InterruptedException ex) {
            System.out.println(ex.getMessage());
        }
        estadoPausa.parar();
        refugio.salirTunel(tunel, id); // sale del tunel y llega a la zona exterior

        Log.escribir(id + " ha salido del refugio por el tunel " + tunel + ".");
        System.out.println(id + " ha salido del refugio por el tunel " + tunel + ".");
        estadoPausa.parar();
    }

    /**
     * Permite que un humano entre en la zona común del refugio, permanezca allí
     * durante un tiempo determinado y luego salga de la zona común. Durante
     * este proceso, se registran mensajes en el log y en la consola para
     * indicar las acciones realizadas.
     *
     * Internamenmte se maneja una InterruptedException para manejar la
```

```
     * interrupción del hilo
     */
    public void irZonaComun() {
        estadoPausa.parar();
        Log.escribir(id + " entra en la zona comun.");
        System.out.println(id + " entra en la zona comun.");
        refugio.irZonaComun(id);
        estadoPausa.parar();
        try {
            sleep((int) ((Math.random() * 1000 + 1000) / Velocidad.getVelocidad())); //en zona
comun 1 a 2 seg
        } catch (InterruptedException ex) {
            System.out.println(ex.getMessage());
        }
        estadoPausa.parar();
    }

    /**
     * Obtiene el identificador único del humano.
     *
     * @return el identificador único (id) como una cadena de texto.
     */
    public String getid() {
        return id;
    }

    /**
     * Establece el identificador único del humano.
     *
     * @param id el nuevo identificador único (id) como una cadena de texto.
     */
    public void setId(String id) {
        this.id = id;
    }

    /**
     * Verifica si el humano está herido.
     *
     * @return {@code true} si el humano está herido, {@code false} en caso
     * contrario.
     */
    public boolean isHerido() {
        return herido;
    }
```

```
/**
 * Establece el estado de herido del humano.
 *
 * @param herido true si el humano está herido, false en caso contrario.
 */
public void setHerido(boolean herido) {
    this.herido = herido;
}

/**
 * Verifica si el humano está muerto.
 *
 * @return {@code true} si el humano está muerto, {@code false} en caso
 * contrario.
 */
public boolean isMuerto() {
    return muerto;
}

/**
 * Establece el estado de muerto del humano.
 *
 * @param muerto true si el humano está muerto, false en caso contrario.
 */
public void setMuerto(boolean muerto) {
    this.muerto = muerto;
}

/**
 * Verifica si el humano está siendo atacado.
 *
 * @return {@code true} si el humano está siendo atacado, {@code false} en
 * caso contrario.
 */
public boolean isSiendoAtacado() {
    return siendoAtacado;
}

/**
 * Establece el estado de si el humano está siendo atacado.
 *
 * @param siendoAtacado true si el humano está siendo atacado, false en caso
 * contrario.
```

```
     */
    public void setSiendoAtacado(boolean siendoAtacado) {
        this.siendoAtacado = siendoAtacado;
    }

    /**
     * Obtiene el refugio asociado al humano.
     * @return el refugio en el que se encuentra el humano.
     */
    public Refugio getRefugio() {
        return refugio;
    }

    /**
     * Establece el refugio asociado al humano.
     * @param refugio el nuevo refugio en el que se encuentra el humano.
     */
    public void setRefugio(Refugio refugio) {
        this.refugio = refugio;
    }

}


public class Zombie extends Thread {
    private String id;
    private int bodycount = 0;
    private ZonaExterior[] zonas = new ZonaExterior[4];
    private Ranking r;
    private EstadoPausa estadoPausa;

    public Zombie(String id, ZonaExterior[] zonas, Ranking r, EstadoPausa estadoPausa) {
        this.id = "Z" + id.substring(1); //id del humano sin la H
        this.zonas = zonas;
        this.r = r;
        this.estadoPausa = estadoPausa;
    }

    /**
     * Método que representa el comportamiento de un hilo de tipo Zombie. El
     * zombie selecciona aleatoriamente una zona exterior para buscar cerebros,
     * realiza acciones como llegar, atacar y abandonar la zona, y luego repite
     * el proceso.
     *
```

```
 * El método incluye pausas controladas por el objeto `estadoPausa` y simula
 * tiempos de espera aleatorios para representar el comportamiento del
 * zombie.
 *
 * Maneja las siguientes excepciones: - InterruptedException: Si el hilo es
 * interrumpido durante su ejecución. - Exception: Para capturar y registrar
 * cualquier otro error inesperado.
 *
 * Las acciones del zombie se registran en el log y se imprimen en la
 * consola.
 */
@Override
public void run() {
    while (true) {
        try {
            int zona = (int) (Math.random() * 4); //seleccionar zona de 0-3

            Log.escribir(id + " esta buscando cerebros en la zona exterior " + zona + ".");
            System.out.println(id + " está buscando cerebros en la zona exterior " + zona + ".");
            zonas[zona].zombieLlegar(this);
            estadoPausa.parar();
            zonas[zona].zombieAtacar(this);
            estadoPausa.parar();
            sleep((int) ((Math.random() * 1000 + 2000) / Velocidad.getVelocidad())); //dormir 2-3 s
            estadoPausa.parar();
            zonas[zona].zombieIrse(this);
            Log.escribir(id + " abandona la zona exterior " + zona + ".");
            System.out.println(id + " abandona la zona exterior " + zona + ".");
            estadoPausa.parar();
            //vuelve a buscar zona

        } catch (InterruptedException ie) {
            Log.escribir(id + " ha sido interrumpido.");
            System.out.println(id + " ha sido interrumpido.");
        } catch (Exception e) {
            Log.escribir("Error en el hilo de " + id + ": " + e.getMessage());
            System.out.println("Error en el hilo de " + id + ": " + e.getMessage());
        }
    }
}

/**
 * Obtiene el identificador único del zombie.
 *
```

```
     * @return el identificador (id) del zombie como una cadena de texto.
     */
    public String getid() {
        return id;
    }

    /**
     * Realiza un ataque de un zombie a un humano.
     *
     * El método simula un ataque de un zombie a un humano. Si el humano es
     * herido, se registra en la pantalla principal. Si el humano es matado, se
     * incrementa el contador de muertes del zombie, se actualiza el ranking y
     * se registra el evento en el log. El tiempo del ataque es aleatorio y
     * depende de la velocidad configurada.
     *
     * @param matado indica si el humano ha sido matado o herido.
     * @param hid el identificador del humano atacado.
     */
    public void atacar(boolean matado, String hid) {
        PantallaPrincipal.getInstancia().addHerido(hid);
        int tiempoAtaque = (int) (Math.random() * 1000 + 500); //0.5-1.5 seg
        try {
            sleep((int) (tiempoAtaque / Velocidad.getVelocidad()));
        } catch (InterruptedException ex) {
            System.out.println(ex.getMessage());
        }
        if (matado) {
            bodycount++;
            r.actualizarRanking(id, bodycount);
            Log.escribir(id + " ha matado al humano " + hid + ". Total muertos: " + bodycount + ".");
            System.out.println(id + " ha matado al humano " + hid + ". Total muertos: " + bodycount
+ ".");
        } else {
            Log.escribir(id + " ha atacado y herido al humano " + hid + ".");
            System.out.println(id + " ha atacado y herido al humano " + hid + ".");
        }
    }

}



public class Refugio {
```

```
    private AtomicInteger comida = new AtomicInteger(0);
    private AtomicInteger humanos = new AtomicInteger(0);
    private AtomicInteger humanosTotales = new AtomicInteger(0); // Total de humanos que han
pasado por el refugio
    private AtomicInteger zombiesTotales = new AtomicInteger(1); // Total de zombies que han
pasado por el refugio

    private AtomicInteger enZonaComun = new AtomicInteger(0);
    private CopyOnWriteArrayList<String> humanosZonaComun = new
CopyOnWriteArrayList<>();

    private AtomicInteger enCama = new AtomicInteger(0);
    private CopyOnWriteArrayList<String> humanosZonaDescanso = new
CopyOnWriteArrayList<>();

    private AtomicInteger enComedor = new AtomicInteger(0);
    private CopyOnWriteArrayList<String> humanosComedor = new CopyOnWriteArrayList<>();

    private Tunel[] tuneles = new Tunel[4]; // Array de túneles
    private Servidor servidor;

    private Lock comidaLock = new ReentrantLock();
    private Condition esperarComida = comidaLock.newCondition();

    /**
     * Constructor de la clase Refugio. Inicializa los túneles y el servidor.
     * @param servidor El servidor asociado al refugio.
     */
    public Refugio(Servidor servidor) {
        for (int i = 0; i < 4; i++) {
            tuneles[i] = new Tunel(i, servidor); // Inicializa cada túnel con su ID y el servidor`
        }
        this.servidor = servidor;
    }

    /**
     * Metodo para aumentar el contador de humanos totales. p Se invoca cuando
     * un humano entra al refugio. Y se actualiza el servidor y la pantalla
     * principal.
     */
    public void unHumanoMas() {
        humanosTotales.incrementAndGet();
        System.out.println("  Un humano más en el mundo... " + humanosTotales.get());
        Log.escribir("  Un humano más en el mundo... " + humanosTotales.get());
```

```
    servidor.actualizarDatosHumanosTotal(humanosTotales.get());
}

/**
 * Reduce en uno el número total de humanos totales. Este método decrementa
 * el contador de humanos totales, imprime un mensaje en la consola
 * indicando la nueva cantidad de humanos y registra esta información en el
 * log del sistema. Además, actualiza los datos del servidor relacionados
 * con el total de humanos.
 */
public void unHumanoMenos() {
    humanosTotales.decrementAndGet();
    System.out.println("  Un humano menos en el mundo... " + humanosTotales.get());
    Log.escribir("  Un humano menos en el mundo... " + humanosTotales.get());
    servidor.actualizarDatosHumanosTotal(humanosTotales.get());
}

/**
 * Reduce en uno el contador total de zombies y actualiza la información
 * relacionada.
 *
 * Este método decrementa el número total de zombies utilizando un contador
 * atómico, imprime un mensaje en la consola indicando la nueva cantidad de
 * zombies, registra el evento en el log del sistema y notifica al servidor
 * para actualizar los datos relacionados con el total de zombies.
 */
public void unZombieMenos() {
    zombiesTotales.decrementAndGet();
    System.out.println("  Un zombie menos en el mundo... " + zombiesTotales.get());
    Log.escribir("  Un zombie menos en el mundo... " + zombiesTotales.get());
    servidor.actualizarDatosZombiesTotal(zombiesTotales.get());
}

/**
 * Aumenta en uno el contador total de zombies y actualiza la información
 * relacionada.
 *
 * Este método incrementa el número total de zombies utilizando un contador
 * atómico, imprime un mensaje en la consola indicando la nueva cantidad de
 * zombies, registra el evento en el log del sistema y notifica al servidor
 * para actualizar los datos relacionados con el total de zombies.
 */
public void unZombieMas() {
    zombiesTotales.incrementAndGet();
```

```
        System.out.println("  Un zombie más en el mundo... " + zombiesTotales.get());
        Log.escribir("  Un zombie más en el mundo... " + zombiesTotales.get());
        servidor.actualizarDatosZombiesTotal(zombiesTotales.get());
    }

    /**
     * Aumenta la cantidad de comida en el refugio.
     *
     * Este método incrementa la cantidad de comida utilizando un contador
     * atómico, imprime un mensaje en la consola indicando la nueva cantidad de
     * comida, registra el evento en el log del sistema y notifica al servidor
     * para actualizar los datos relacionados con la comida.
     *
     * @param comidaExtra La cantidad de comida a agregar al refugio.
     */
    public void addComida(int comidaExtra) {
        comidaLock.lock();
        try {
            int c = comida.addAndGet(comidaExtra);
            System.out.println("  Llegan raciones de refugios aliados: " + comidaExtra + " de
comida!! Ahora hay: " + c);
            Log.escribir("  Llegan raciones de refugios aliados: " + comidaExtra + " de comida!!  " +
c);
            PantallaPrincipal.getInstancia().actualizarComida(c);
            servidor.actualizarDatosComida(c);
            esperarComida.signalAll();
        } finally {
            comidaLock.unlock();
        }
    }

    /**
     * Aumenta la cantidad de comida en el refugio.
     *
     * Este método incrementa la cantidad de comida en 2 utilizando un contador
     * atómico, imprime un mensaje en la consola indicando la nueva cantidad de
     * comida, registra el evento en el log del sistema y notifica al servidor
     * para actualizar los datos relacionados con la comida. También notifica a
     * los hilos en espera para que puedan continuar su ejecución.
     */
    public void dejarComida() {
        comidaLock.lock();
        try {
            int c = comida.addAndGet(2);
```

```
            System.out.println("  dejan 2 de comida!! " + c);
            PantallaPrincipal.getInstancia().actualizarComida(c);
            servidor.actualizarDatosComida(c);
            esperarComida.signalAll();
        } finally {
            comidaLock.unlock();
        }
    }

    /**
     * Método que registra la entrada de un humano al refugio.
     *
     * Incrementa el contador de humanos en el refugio, actualiza la interfaz
     * gráfica para reflejar el nuevo número de humanos y notifica al servidor
     * sobre el cambio en los datos del refugio. Finalmente, imprime un mensaje
     * en la consola indicando que el humano ha entrado al refugio junto con el
     * número actual de humanos.
     *
     * @param id Identificador del humano que entra al refugio. Este
     * identificador se utiliza para registrar y mostrar información sobre la
     * entrada del humano.
     */
    public void humanoEntraRefugio(String id) {
        humanos.incrementAndGet();
        PantallaPrincipal.getInstancia().actualizarHumanos(humanos.get());
        servidor.actualizarDatosRefugio(humanos.get());
        System.out.println(id + " entra al refugio..." + humanos.get());
    }

    /**
     * Método que registra la salida de un humano del refugio.
     *
     * Este método decrementa el contador de humanos en el refugio, actualiza la
     * interfaz gráfica para reflejar el nuevo número de humanos y notifica al
     * servidor sobre el cambio en los datos del refugio. Finalmente, imprime un
     * mensaje en la consola indicando que el humano ha salido del refugio junto
     * con el número actual de humanos.
     *
     * @param id Identificador del humano que sale del refugio. Este
     * identificador se utiliza para registrar y mostrar información sobre la
     * salida del humano.
     */
    public void humanoSaleRefugio(String id) {
        humanos.decrementAndGet();
```

```
    PantallaPrincipal.getInstancia().actualizarHumanos(humanos.get());
    servidor.actualizarDatosRefugio(humanos.get());
    System.out.println(id + " sale del refugio..." + humanos.get());
}

/**
 * Método que registra la entrada de un humano a la zona común del refugio.
 *
 * Este método incrementa el contador de humanos en la zona común, actualiza
 * la interfaz gráfica para reflejar el nuevo número de humanos en la zona
 * común.
 *
 * @param id Identificador del humano que entra a la zona común. Este
 * identificador se utiliza para registrar y mostrar información sobre la
 * entrada del humano.
 */
public void irZonaComun(String id) {
    enZonaComun.incrementAndGet();
    humanosZonaComun.add(id);
    PantallaPrincipal.getInstancia().actualizarZonaComun(humanosZonaComun);
}

/**
 * Método que registra la salida de un humano de la zona común del refugio.
 *
 * Este método decrementa el contador de humanos en la zona común, actualiza
 * la interfaz gráfica para reflejar el nuevo número de humanos en la zona
 * común.
 *
 * @param id Identificador del humano que sale de la zona común. Este
 * identificador se utiliza para registrar y mostrar información sobre la
 * salida del humano.
 */
public void salirZonaComun(String id) {
    enZonaComun.decrementAndGet();
    humanosZonaComun.remove(id);
    PantallaPrincipal.getInstancia().actualizarZonaComun(humanosZonaComun);
}

/**
 * Método que registra la entrada de un humano a la zona de descanso del
 * refugio.
 *
 * Este método incrementa el contador de humanos en la zona de descanso,
```

```
     * actualiza la interfaz gráfica para reflejar el nuevo número de humanos en
     * la zona de descanso.
     *
     * @param id Identificador del humano que entra a la zona de descanso. Este
     * identificador se utiliza para registrar y mostrar información sobre la
     * entrada del humano.
     */
    public void irZonaDescanso(String id) {
        enCama.incrementAndGet();
        humanosZonaDescanso.add(id);
        PantallaPrincipal.getInstancia().actualizarZonaDescanso(humanosZonaDescanso);
    }

    /**
     * Método que registra la salida de un humano de la zona de descanso del
     * refugio.
     *
     * Este método decrementa el contador de humanos en la zona de descanso,
     * actualiza la interfaz gráfica para reflejar el nuevo número de humanos en
     * la zona de descanso.
     *
     * @param id Identificador del humano que sale de la zona de descanso. Este
     * identificador se utiliza para registrar y mostrar información sobre la
     * salida del humano.
     */
    public void salirZonaDescanso(String id) {
        enCama.decrementAndGet();
        humanosZonaDescanso.remove(id);
        PantallaPrincipal.getInstancia().actualizarZonaDescanso(humanosZonaDescanso);
    }

    /**
     * Método que registra la entrada de un humano al comedor del refugio.
     *
     * Este método sincronizado incrementa el contador de humanos en el comedor,
     * actualiza la interfaz gráfica para reflejar el nuevo número de humanos en
     * el comedor y notifica a los demás hilos si hay comida disponible. Si no
     * hay comida, el hilo se bloquea hasta que haya comida disponible.
     *
     * @param id Identificador del humano que entra al comedor. Este
     * identificador se utiliza para registrar y mostrar información sobre la
     * entrada del humano.
     */
    public void irComedor(String id) {
```

```
    comidaLock.lock();
    try {
        System.out.println(id + " llega al comedor...Queda: " + comida.get());
        enComedor.incrementAndGet();
        humanosComedor.add(id);
        PantallaPrincipal.getInstancia().actualizarComedor(humanosComedor);
        while (comida.get() == 0) {
            System.out.println(id + " esta esperando por comida... Queda: " + comida.get());
            try {
                esperarComida.await();
            } catch (InterruptedException ex) {
                System.out.println(ex.getMessage());
            }
        }
        int c = comida.decrementAndGet(); //comer 1 comida
        System.out.println(id + " come... Queda: " + c);
        PantallaPrincipal.getInstancia().actualizarComida(c);
        servidor.actualizarDatosComida(c);

    } finally {
        comidaLock.unlock();
    }

}

/**
 * Método que registra la salida de un humano del comedor del refugio.
 *
 * Este método decrementa el contador de humanos en el comedor y actualiza
 * la interfaz gráfica para reflejar el nuevo número de humanos en el
 * comedor.
 *
 * @param id Identificador del humano que sale del comedor. Este
 * identificador se utiliza para registrar y mostrar información sobre la
 * salida del humano.
 */
public void salirComedor(String id) {
    enComedor.decrementAndGet();
    humanosComedor.remove(id);
    PantallaPrincipal.getInstancia().actualizarComedor(humanosComedor);
}

/**
 * Permite que un humano salga del refugio a través de un túnel específico.
```

```
     *
     * Este método realiza las siguientes acciones en orden: 1. El humano sale
     * de la zona común del refugio. 2. Se actualiza el contador de humanos
     * dentro del refugio. 3. El humano sale del refugio a través del túnel
     * especificado.
     *
     * @param tunel El índice del túnel por el cual el humano saldrá. Debe estar
     * dentro del rango válido de túneles (0 <= túnel < tuneles.length).
     * @param idHumano El identificador único del humano que saldrá del refugio.
     * @throws IllegalArgumentException Si el índice del túnel es inválido.
     */
    public void salirRefugio(int tunel, String idHumano) {
        if (tunel >= 0 && tunel < tuneles.length) {
            // Si cambias el orden da las dos siguientes líneas, el humano aparece como que sale o
no de la zona común para ir al túnel
            salirZonaComun(idHumano); // Salir de la zona común al salir del refugio
            humanoSaleRefugio(idHumano); // disminuir el contador de humanos en el refugio
            tuneles[tunel].salirRefugio(idHumano); // Salir del refugio y entrar
        } else {
            throw new IllegalArgumentException("Túnel inválido: " + tunel);
        }
    }

    /**
     * Permite que un humano entre al refugio a través de un túnel específico.
     *
     * @param tunel El índice del túnel por el cual el humano entrará. Debe
     * estar dentro del rango válido de túneles (0 <= túnel < tuneles.length).
     * @param idHumano El identificador único del humano que entrará al refugio.
     * @throws IllegalArgumentException Si el índice del túnel es inválido.
     */
    public void entrarRefugio(int tunel, String idHumano) {
        if (tunel >= 0 && tunel < tuneles.length) {
            tuneles[tunel].entrarRefugio(idHumano);

        } else {
            throw new IllegalArgumentException("Túnel inválido: " + tunel);
        }
    }

    /**
     * Permite que un humano salga de un túnel específico.
     *
     * @param tunel El índice del túnel por el cual el humano saldrá. Debe estar
```

```
     * dentro del rango válido de túneles (0 <= túnel < tuneles.length).
     * @param idHumano El identificador único del humano que saldrá del túnel.
     * @throws IllegalArgumentException Si el índice del túnel es inválido.
     */
    public void salirTunel(int tunel, String idHumano) {
        if (tunel >= 0 && tunel < tuneles.length) {
            tuneles[tunel].salirTunel(idHumano);
        } else {
            throw new IllegalArgumentException("Túnel inválido: " + tunel);
        }
    }

}

public class Tunel {
    private int humanosDentro = 0;
    private Lock entrando = new ReentrantLock();
    private Condition quiereEntrar = entrando.newCondition();
    private Condition quiereSalir = entrando.newCondition();
    private int humanosEntrando = 0;
    private CopyOnWriteArrayList<String> humanosSaliendoRef = new
CopyOnWriteArrayList<>();
    private CopyOnWriteArrayList<String> humanosEntrandoRef = new
CopyOnWriteArrayList<>();
    private CopyOnWriteArrayList<String> humanosDentroRef = new CopyOnWriteArrayList<>();
    private CyclicBarrier barrera = new CyclicBarrier(3);
    private int id;
    private Servidor servidor;

    /**
     * Constructor de la clase Tunel.
     *
     * @param id
     * @param servidor
     */
    public Tunel(int id, Servidor servidor) {
        this.servidor = servidor;
        this.id = id;
    }

    /**
     * Actualiza el número de humanos presentes en el túnel y notifica al
     * servidor. Este método calcula la cantidad total de humanos que están
     * entrando, dentro y saliendo del túnel, y actualiza esta información en el
```

```
     * servidor.
     *
     * Nota: No se utiliza la palabra clave synchronized en este método, ya que
     * las variables involucradas (humanosSaliendoRef, humanosDentroRef y
     * humanosEntrandoRef) son atómicas, y el método llamado en el servidor
     * (actualizarDatosTuneles) ya está sincronizado.
     */
    public void actualizarHumanosEnTunel() {
        servidor.actualizarDatosTuneles(id, humanosSaliendoRef.size() +
humanosDentroRef.size() + humanosEntrandoRef.size());
    }

    /**
     * Permite que un humano salga del refugio y actualiza el estado del túnel y
     * del refugio.
     *
     * Este método realiza las siguientes acciones: 1. Añade el identificador
     * del humano a la lista de humanos que están saliendo del refugio. 2.
     * Actualiza la interfaz gráfica para reflejar los cambios en el túnel. 3.
     * Espera a que la barrera se complete (cuando tres hilos alcanzan el punto
     * de sincronización). 4. Bloquea el acceso para garantizar que no haya
     * interferencias mientras se actualizan los estados. 5. Espera hasta que no
     * haya humanos dentro del túnel o entrando al túnel. 6. Incrementa el
     * contador de humanos dentro del túnel y actualiza las listas
     * correspondientes. 7. Actualiza la interfaz gráfica para reflejar los
     * cambios en el túnel y el refugio.
     *
     * @param idHumano El identificador único del humano que desea salir del
     * refugio.
     */
    public void salirRefugio(String idHumano) {
        humanosSaliendoRef.add(idHumano);
        actualizarHumanosEnTunel();
        PantallaPrincipal.getInstancia().actualizarTunel(id, humanosSaliendoRef);
        try {
            barrera.await();
        } catch (InterruptedException | BrokenBarrierException ex) {
            System.out.println(ex);
        }
        //cuando ya pasan 3 la barrera
        entrando.lock();
        try {
            while (humanosDentro > 0 || humanosEntrando > 0) {
                try {
```

```
                quiereSalir.await();
            } catch (InterruptedException ex) {
                System.out.println(ex);
            }
        }
        humanosDentro++;
        humanosSaliendoRef.remove(idHumano); //eliminamos el id del humano de la lista de
ids
        humanosDentroRef.add(idHumano); //añadimos el id del humano a la lista de ids
        PantallaPrincipal.getInstancia().actualizarTunel(id, humanosSaliendoRef); //actualizo la
pantalla del tunel
        PantallaPrincipal.getInstancia().actualizarTunelMedio(id, humanosDentroRef);
//actualizo la pantalla del refugio
    } finally {
        entrando.unlock();
    }
}

/**
 * Método que permite a un humano intentar entrar al refugio a través del
 * túnel.
 *
 * Este método realiza las siguientes acciones: - Bloquea el acceso
 * concurrente al túnel utilizando un mecanismo de bloqueo. - Incrementa el
 * contador de humanos intentando entrar y añade el identificador del humano
 * a la lista de IDs de humanos entrando. - Actualiza la interfaz gráfica
 * para reflejar los cambios en el estado del túnel. - Espera hasta que no
 * haya humanos dentro del túnel antes de permitir la entrada. - Una vez
 * permitido, decrementa el contador de humanos entrando, incrementa el
 * contador de humanos dentro y actualiza las listas correspondientes. -
 * Actualiza nuevamente la interfaz gráfica para reflejar los cambios en el
 * estado del túnel y el refugio.
 *
 * Este método garantiza la sincronización adecuada para evitar condiciones
 * de carrera y asegurar que los humanos entren al refugio de manera
 * controlada.
 *
 * @param idHumano El identificador único del humano que desea entrar al
 * refugio.
 */
public void entrarRefugio(String idHumano) {
    entrando.lock();
    try {
        humanosEntrando++;
```

```
            humanosEntrandoRef.add(idHumano); //añadimos el id del humano a la lista de ids
            actualizarHumanosEnTunel();
            PantallaPrincipal.getInstancia().actualizarTunelFuera(id, humanosEntrandoRef);
//actualizo la pantalla del tunel

            while (humanosDentro > 0) {
                try {
                    quiereEntrar.await();
                } catch (InterruptedException ex) {
                    System.out.println(ex);
                }
            }
            humanosEntrando--;
            humanosDentro++;

            humanosEntrandoRef.remove(idHumano); //eliminamos el id del humano de la lista de
ids
            humanosDentroRef.add(idHumano); //añadimos el id del humano a la lista de ids
            PantallaPrincipal.getInstancia().actualizarTunelFuera(id, humanosEntrandoRef);
//actualizo la pantalla del tunel
            PantallaPrincipal.getInstancia().actualizarTunelMedio(id, humanosDentroRef);
//actualizo la pantalla del refugio
        } finally {
            entrando.unlock();
        }
    }

    /**
     * Método que permite a un humano salir del túnel.
     *
     * Este método realiza las siguientes acciones: - Disminuye el contador de
     * humanos dentro del túnel. - Elimina el identificador del humano de la
     * lista de referencias de humanos dentro del túnel. - Actualiza la
     * información visual del túnel en la pantalla principal, tanto en la
     * sección del refugio como en la sección del túnel. - Si hay humanos
     * esperando para entrar al túnel, les da prioridad y les notifica para que
     * puedan entrar. - Si no hay humanos esperando para entrar, permite que los
     * humanos que desean salir continúen haciéndolo.
     *
     * Este método utiliza un mecanismo de bloqueo para garantizar la
     * sincronización entre los hilos que interactúan con el túnel.
     *
     * @param idHumano El identificador único del humano que desea salir del
     * túnel.
```

```
     */
    public void salirTunel(String idHumano) {
        entrando.lock();
        try {
            humanosDentro--;
            humanosDentroRef.remove(idHumano); //eliminamos el id del humano de la lista de ids
            actualizarHumanosEnTunel();
            PantallaPrincipal.getInstancia().actualizarTunelMedio(id, humanosDentroRef);
//actualizo la pantalla del refugio
            PantallaPrincipal.getInstancia().actualizarTunelFuera(id, humanosEntrandoRef);
//actualizo la pantalla del tunel
            if (humanosEntrando > 0) {
                // Si siguen quedando humanos intentando entrar, priorizamos su entrada
                quiereEntrar.signalAll();
            } else {
                // Si ya no quedan, permitimos que los que salen sigan
                quiereSalir.signalAll();
            }
        } finally {
            entrando.unlock();
        }
    }

}



public class ZonaExterior {

    private AtomicInteger ctdzombies = new AtomicInteger(0);
    private AtomicInteger ctdhumanos = new AtomicInteger(0);
    private int id;
    private Servidor servidor;

    private CopyOnWriteArrayList<Humano> humanos = new CopyOnWriteArrayList<>();
    private CopyOnWriteArrayList<Zombie> zombies = new CopyOnWriteArrayList<>();

    private CopyOnWriteArrayList<String> listaIDsHumanos = new CopyOnWriteArrayList<>(); //
IDs de los humanos y zombies en la zona exterior
    private CopyOnWriteArrayList<String> listaIDsZombies = new CopyOnWriteArrayList<>(); //
IDs de los humanos y zombies en la zona exterior

    /**
     * Constructor de la clase ZonaExterior.
```

```
     *
     * @param id Identificador único de la zona exterior.
     * @param servidor Instancia del servidor asociada a la zona exterior.
     */
    public ZonaExterior(int id, Servidor servidor) {
        this.id = id;
        this.servidor = servidor;
    }

    /**
     * Método que gestiona la llegada de un zombie a la zona exterior.
     *
     * Este método realiza las siguientes acciones: - Incrementa el contador de
     * zombies en la zona exterior. - Añade el ID del zombie a la lista de IDs
     * de zombies presentes en la zona. - Agrega el objeto Zombie a la lista de
     * zombies. - Actualiza la interfaz gráfica de la pantalla principal para
     * reflejar los cambios en los zombies presentes en la zona exterior. -
     * Notifica al servidor para actualizar los datos de capacidad de la zona de
     * riesgo.
     *
     * @param z El objeto Zombie que está llegando a la zona exterior.
     */
    public void zombieLlegar(Zombie z) {
        ctdzombies.incrementAndGet();
        listaIDsZombies.add(z.getid()); //añadimos el id del zombie a la lista de ids
        zombies.add(z);
        PantallaPrincipal.getInstancia().actualizarExteriorZombies(id, listaIDsZombies); //actualizo
la pantalla de la zona exterior
        servidor.actualizarDatosZombiesZonaRiesgo(id, ctdzombies.get()); //actualizo la capacidad
de la zona exterior
    }

    /**
     * Método sincronizado que permite a un zombie atacar a un humano en la zona
     * exterior. Si un humano es atacado, se marca como herido o muerto, y se
     * elimina de la lista de humanos disponibles para evitar ataques repetidos.
     * Además, se actualizan las interfaces y datos relacionados con la zona
     * exterior.
     *
     * Detalles del proceso: - Si hay humanos disponibles, se selecciona uno al
     * azar. - El humano seleccionado es marcado como "siendo atacado". - Existe
     * una probabilidad de 1/3 de que el humano muera; si no, será marcado como
     * herido. - El humano atacado es eliminado de la lista de humanos y su ID
     * es removido de la lista de IDs. - Se notifica al zombie sobre el
```

```
     * resultado del ataque (muerte o supervivencia del humano). - Se actualizan
     * las interfaces gráficas y los datos del servidor para reflejar los
     * cambios.
     *
     * Nota: Este método asume que un humano herido no puede ser atacado
     * nuevamente, por lo que es eliminado inmediatamente después de ser
     * atacado.
     *
     * @param z El objeto Zombie que está realizando el ataque.
     */
    public synchronized void zombieAtacar(Zombie z) {
        if (ctdhumanos.get() > 0) {
            int humano = (int) (Math.random() * ctdhumanos.get()); //humano en el array en la
posicion "humano"
            Humano h = humanos.get(humano);

            h.setSiendoAtacado(true); //marcar que esta siendo atacado

            boolean muerte = false;  //inicializamos a que el humano sobrevive
            if (Math.random() > 0.66) { //1/3 de morir
                muerte = true; //si se cumple, el humano muere y se transforma
                h.setMuerto(true);
            } else {
                h.setHerido(true);
            }
            //el humano ha sido atacadoa. Se borra para que otro zombie no le ataque
            h.interrupt();
            ctdhumanos.decrementAndGet();
            listaIDsHumanos.remove(h.getid()); //eliminamos el id del humano de la lista de ids
            humanos.remove(h);

            z.atacar(muerte, h.getid()); //Se le pasa el id del humano para los logs
            h.setSiendoAtacado(false);
            PantallaPrincipal.getInstancia().actualizarExteriorHumanos(id, listaIDsHumanos);
//actualizo la pantalla de la zona exterior
            servidor.actualizarDatosHumanosZonaRiesgo(id, ctdhumanos.get()); //actualizo la
capacidad de la zona exterior

            notifyAll();
        }
    }

    /**
     * Método que gestiona la salida de un zombie de la zona exterior.
```

```
     *
     * Este método realiza las siguientes acciones: - Decrementa el contador de
     * zombies en la zona exterior. - Elimina el ID del zombie de la lista de
     * IDs de zombies presentes en la zona. - Elimina el objeto Zombie de la
     * lista de zombies. - Actualiza la interfaz gráfica de la pantalla
     * principal para reflejar los cambios en los zombies presentes en la zona
     * exterior. - Notifica al servidor para actualizar los datos de capacidad
     * de la zona de riesgo.
     *
     * @param z El objeto Zombie que está saliendo de la zona exterior.
     */
    public void zombieIrse(Zombie z) {
        ctdzombies.decrementAndGet();
        listaIDsZombies.remove(z.getid()); //eliminamos el id del zombie de la lista de ids
        zombies.remove(z);
        PantallaPrincipal.getInstancia().actualizarExteriorZombies(id, listaIDsZombies); //actualizo
la pantalla de la zona exterior
        servidor.actualizarDatosZombiesZonaRiesgo(id, ctdzombies.get()); //actualizo la capacidad
de la zona exterior
    }

    /**
     * Método que gestiona la llegada de un humano a la zona exterior.
     *
     * Este método realiza las siguientes acciones: - Incrementa el contador de
     * humanos en la zona exterior. - Añade el ID del humano a la lista de IDs
     * de humanos presentes en la zona. - Agrega el objeto Humano a la lista de
     * humanos. - Actualiza la interfaz gráfica de la pantalla principal para
     * reflejar los cambios en los humanos presentes en la zona exterior. -
     * Notifica al servidor para actualizar los datos de capacidad de la zona de
     * riesgo.
     *
     * @param h El objeto Humano que está llegando a la zona exterior.
     */
    public void humanoLlegar(Humano h) {
        ctdhumanos.incrementAndGet();
        listaIDsHumanos.add(h.getid()); //añadimos el id del humano a la lista de ids
        humanos.add(h);
        PantallaPrincipal.getInstancia().actualizarExteriorHumanos(id, listaIDsHumanos);
//actualizo la pantalla de la zona exterior
        servidor.actualizarDatosHumanosZonaRiesgo(id, ctdhumanos.get()); //actualizo la
capacidad de la zona exterior
    }
```

```
    /**
     * Permite que un humano abandone la zona exterior si cumple con ciertas
     * condiciones.
     *
     * El método realiza las siguientes acciones: - Verifica que el humano no
     * esté muerto, herido ni siendo atacado. - Si cumple las condiciones,
     * decrementa el contador de humanos en la zona. - Elimina el ID del humano
     * de la lista de IDs y lo remueve de la lista de humanos. - Actualiza la
     * interfaz gráfica para reflejar los cambios en la zona exterior. -
     * Notifica al servidor para actualizar los datos de la zona de riesgo.
     *
     * Si el humano está siendo atacado, se llama al método `humanoAtacado(h)`.
     *
     * @param h El objeto Humano que está abandonando la zona exterior.
     */
    public void humanoIrse(Humano h) {
        if (!h.isMuerto() && !h.isHerido() && !h.isSiendoAtacado()) {
            ctdhumanos.decrementAndGet();
            listaIDsHumanos.remove(h.getid()); //eliminamos el id del humano de la lista de ids
            humanos.remove(h);
            PantallaPrincipal.getInstancia().actualizarExteriorHumanos(id, listaIDsHumanos);
//actualizo la pantalla de la zona exterior
            servidor.actualizarDatosHumanosZonaRiesgo(id, ctdhumanos.get()); //actualizo la
capacidad de la zona exterior
        } else if (h.isSiendoAtacado()) {
            humanoAtacado(h);
        }
    }

    /**
     * Método que permite a un humano esperar si está siendo atacado.
     *
     * Este método utiliza un bucle `while` para verificar si el humano está
     * siendo atacado. Si es así, el hilo se pone en espera hasta que se
     * notifique que el ataque ha finalizado.
     * @param h el humano que debe esperar.
     */
    public synchronized void humanoAtacado(Humano h) {
        while (h.isSiendoAtacado()) {
            try {
                wait();
            } catch (InterruptedException ex) {
                System.out.println(ex.getMessage());
            }
```

```
        }
    }

}
```

```
public class Cliente {
    private Socket socket;
    private PrintWriter salida;
    private BufferedReader entrada;
    private boolean conectado = false;

    /**
     * Establece una conexión con un servidor utilizando un socket. Este método
     * intenta crear un socket para conectarse al servidor especificado en el
     * host y puerto proporcionados. Si la conexión es exitosa, se inicializan
     * los streams de entrada y salida para la comunicación con el servidor. En
     * caso de error, se captura la excepción y se imprime un mensaje de error.
     *
     * @param host La dirección del servidor al que se desea conectar.
     * @param puerto El número de puerto en el que el servidor está escuchando.
     */
    public void conectarAlServidor(String host, int puerto) {
        //Apertura de sockets (en la parte cliente)
        try {
            // Crear un socket para conectarse al servidor
            socket = new Socket(host, puerto);
            System.out.println("Conectado al servidor en " + host + ":" + puerto);

            //Creación de streams de entrada (en cliente)
            entrada = new BufferedReader(new InputStreamReader(socket.getInputStream()));
            salida = new PrintWriter(socket.getOutputStream(), true);
            conectado = true;

        } catch (IOException e) {
            System.out.println("Error al conectar al servidor: " + e.getMessage());
        } catch (Exception e) {
```

```
            System.out.println("Error al conectar al servidor: " + e.getMessage());
        }
    }

    /**
     * Inicia un hilo para la recepción de datos desde el servidor. Este método
     * crea un hilo que escucha continuamente los mensajes enviados por el
     * servidor a través de un socket. Dependiendo del contenido del mensaje, se
     * realizan diferentes acciones, como actualizar datos en la interfaz de
     * usuario o cerrar la conexión si se recibe un mensaje de salida.
     *
     * Comportamiento: - Si el mensaje recibido es "SALIR", se cierran las
     * conexiones y se detiene la aplicación. - Si el mensaje comienza con
     * "data", se interpreta como un conjunto de datos clave-valor y se
     * actualizan los elementos correspondientes en la interfaz de usuario. - Si
     * el formato del mensaje no es reconocido, se imprime un mensaje de error
     * en la consola.
     *
     * Excepciones: - Captura y maneja IOException en caso de errores de
     * entrada/salida durante la recepción de datos.
     *
     * Notas: - Este método utiliza un hilo separado para evitar bloquear el
     * hilo principal de la aplicación. - Asegúrese de que las instancias de
     * ClientePantalla estén correctamente inicializadas antes de llamar a este
     * método.
     */
    public void iniciarRecepcion() {
        Thread receptor = new Thread(() -> {
            System.out.println("Comienza la recepción");
            try {
                String linea;
                while ((linea = entrada.readLine()) != null) {   // cambia null por un mensaje de parada
del hilo al cerrar el socket o servidor
                    if (linea.equalsIgnoreCase("SALIR")) {
                        salida.close();
                        entrada.close();
                        socket.close();
                        System.out.println("Conexion detenida por el servidor.");
                        System.exit(0);

                    } else if (linea.length() >= 4 && linea.substring(0, 4).equals("data")) {
                        String datos = linea.substring(4);

                        if (datos.contains("=")) {
```

```
String[] kv = datos.split("=", 2);
String clave = kv[0];
String valor = kv[1];

if (clave.equals("numHumRefu")) {
    ClientePantalla.getInstancia().actualizarRefugio(valor);
} else if (clave.equals("numHumTotal")) {
    ClientePantalla.getInstancia().actualizarTotalHumanos(valor);
} else if (clave.equals("numZomTotal")) {
    ClientePantalla.getInstancia().actualizarTotalZombies(valor);
} else if (clave.equals("comida")) {
    ClientePantalla.getInstancia().actualizarComida(valor);

} else if (clave.equals("numHumT0")) {
    ClientePantalla.getInstancia().actualizarTunel(0, valor);
} else if (clave.equals("numHumT1")) {
    ClientePantalla.getInstancia().actualizarTunel(1, valor);
} else if (clave.equals("numHumT2")) {
    ClientePantalla.getInstancia().actualizarTunel(2, valor);
} else if (clave.equals("numHumT3")) {
    ClientePantalla.getInstancia().actualizarTunel(3, valor);

} else if (clave.equals("numHumZR0")) {
    ClientePantalla.getInstancia().actualizarExteriorHumanos(0, valor);
} else if (clave.equals("numHumZR1")) {
    ClientePantalla.getInstancia().actualizarExteriorHumanos(1, valor);
} else if (clave.equals("numHumZR2")) {
    ClientePantalla.getInstancia().actualizarExteriorHumanos(2, valor);
} else if (clave.equals("numHumZR3")) {
    ClientePantalla.getInstancia().actualizarExteriorHumanos(3, valor);

} else if (clave.equals("numZomZR0")) {
    ClientePantalla.getInstancia().actualizarExteriorZombies(0, valor);
} else if (clave.equals("numZomZR1")) {
    ClientePantalla.getInstancia().actualizarExteriorZombies(1, valor);
} else if (clave.equals("numZomZR2")) {
    ClientePantalla.getInstancia().actualizarExteriorZombies(2, valor);
} else if (clave.equals("numZomZR3")) {
    ClientePantalla.getInstancia().actualizarExteriorZombies(3, valor);

} else if (clave.equals("ranking1")) {
    ClientePantalla.getInstancia().actualizarRanking(1, valor);
} else if (clave.equals("ranking2")) {
    ClientePantalla.getInstancia().actualizarRanking(2, valor);
```

```
                    } else if (clave.equals("ranking3")) {
                        ClientePantalla.getInstancia().actualizarRanking(3, valor);
                    } else {
                        System.out.println("Clave desconocida: " + clave);
                    }
                } else {
                    System.out.println("Formato de datos incorrecto: " + datos);
                }

            }
        }
    } catch (IOException e) {
        System.out.println("Cerrado");
    }
}
);
receptor.start();
}

/**
 * Método para comprobar si el cliente está conectado al servidor.
 *
 * @return true si el cliente está conectado, false en caso contrario.
 */
public boolean getConectado() {
    return conectado;
}

/**
 * Envía un comando al servidor a través de la salida establecida. Si la
 * salida no es nula, escribe el comando en el flujo de salida y muestra un
 * mensaje en la consola indicando que el comando ha sido enviado.
 *
 * @param comando El comando que se desea enviar al servidor.
 */
public void enviarComando(String comando) {
    if (salida != null) {
        salida.println(comando);
        System.out.println("Comando enviado desde cliente: " + comando);
    }
}

/**
 * Cierra la conexión del cliente con el servidor. Este método envía un
```

```
     * comando de salida al servidor y cierra los streams de entrada y salida,
     * así como el socket. Si el socket es nulo, se imprime un mensaje indicando
     * que no se puede desconectar.
     *
     * @throws IOException Si ocurre un error al cerrar los streams o el socket.
     */
    public void desconectar() throws IOException {
        if (socket != null) {
            enviarComando("SALIR");
            salida.close();
            entrada.close();
            socket.close();
            System.out.println("Cliente desconectado.");
        } else {
            System.out.println("Socket null.");
        }

    }
}

public class Servidor {
    Socket clienteSocket;
    private PrintWriter salida;
    private BufferedReader entrada;
    private EstadoPausa estadoPausa;
    private ServerSocket serverSocket;
    private boolean conectado = false;
    private Refugio refugio;

    /**
     * Establece el estado de pausa del servidor.
     *
     * @param estadoPausa El nuevo estado de pausa que se desea asignar.
     */
    public void setEstadopausa(EstadoPausa estadoPausa) {
        this.estadoPausa = estadoPausa;
    }

    /**
     * Devuelve el estado de conexión del servidor.
     *
     * @return true si el servidor está conectado, false en caso contrario.
     */
    public boolean getConectado() {
```

```
        return conectado;
    }

    /**
     * Establece el refugio asociado al servidor.
     *
     * @param refugio El refugio que se desea asociar al servidor.
     */
    public void setRefugio(Refugio refugio) {
        this.refugio = refugio;
    }

    /**
     * Inicia el servidor en el puerto especificado y espera conexiones de
     * clientes. Una vez conectado un cliente, permite la comunicación
     * bidireccional y escucha comandos enviados por el cliente para controlar
     * el estado del servidor.
     *
     * Comandos soportados por el cliente: - "PAUSAR": Pausa el estado del
     * servidor. - "REANUDAR": Reanuda el estado del servidor. -
     * "VELOCIDAD=<valor>": Cambia la velocidad a un valor específico. -
     * "COMIDA_EXTRA=<valor>": Añade una cantidad específica de comida al
     * refugio. - "SALIR": Detiene el servidor y cierra la conexión con el
     * cliente.
     *
     * Manejo de excepciones: - Si ocurre un error al abrir el socket o al
     * comunicarse con el cliente, se imprime el error en la consola.
     *
     * Notas: - El servidor imprime información sobre su estado y los comandos
     * recibidos en la consola. - Si el cliente envía un comando desconocido, se
     * notifica en la consola. - El servidor se detiene si el cliente envía el
     * comando "SALIR".
     *
     * @param puerto El puerto en el que el servidor escuchará las conexiones de
     * clientes.
     */
    public void iniciarServidor(int puerto) {
        // Iniciar el servidor y esperar conexiones de clientes
        //Apertura de Sockets (en la parte del servidor)
        try { //Creamos un objeto ServerSocket para que esté atento a las conexiones de clientes
potenciales

            // Imprimir el puerto y la dirección IP del servidor para la conexión del cliente
            //Crear objeto Socket para poder enviar y recibir datos
```

```
            serverSocket = new ServerSocket(puerto);
            System.out.println("Servidor iniciado en: " +
serverSocket.getInetAddress().getHostAddress() + ":" + puerto);
            System.out.println("Esperando conexión del cliente...");
            clienteSocket = serverSocket.accept();

            System.out.println("Cliente conectado.");
            conectado = true;

            salida = new PrintWriter(clienteSocket.getOutputStream(), true);
            entrada = new BufferedReader(new
InputStreamReader(clienteSocket.getInputStream()));

            // Hilo para escuchar comandos del cliente
            Thread comandos = new Thread(() -> {
                try {
                    String linea;
                    while ((linea = entrada.readLine()) != null) {
                        if (linea.equalsIgnoreCase("PAUSAR")) {
                            estadoPausa.pausar();
                            System.out.println("Pausado por cliente.");
                        } else if (linea.equalsIgnoreCase("REANUDAR")) {
                            estadoPausa.reanudar();
                            System.out.println("Reanudado por cliente.");
                        } else if (linea.startsWith("VELOCIDAD=")) {
                            try {
                                int v = Integer.parseInt(linea.split("=")[1]);
                                Velocidad.setVelocidad(v);
                                System.out.println("Velocidad cambiada a " + v + " por cliente.");
                            } catch (NumberFormatException e) {
                                System.out.println("Velocidad inválida.");
                            }
                        } else if (linea.startsWith("COMIDA_EXTRA=")) {
                            try {
                                int comida = Integer.parseInt(linea.split("=")[1]);
                                refugio.addComida(comida);
                            } catch (NumberFormatException e) {
                                System.out.println("Comida inválida.");
                            }
                        } else if (linea.equalsIgnoreCase("SALIR")) {
                            salida.close();
                            entrada.close();
                            clienteSocket.close();
                            serverSocket.close();
```

```
                    conectado = false;
                    System.out.println("Servidor detenido por el cliente.");
                    System.out.println("Cliente desconectado.");
                    System.exit(0);
                    break;
                } else {
                    System.out.println("Comando desconocido: " + linea);
                }
            }
        } catch (IOException e) {
            System.out.println("Cerrado");
        }
    });
    comandos.start();

  } catch (IOException e) {
    System.out.println(e);
  }
}

/**
 * Actualiza los datos del refugio enviando el número de humanos presentes.
 *
 * @param numHumanos El número de humanos en el refugio que se desea
 * actualizar.
 */
public void actualizarDatosRefugio(int numHumanos) {
  salida.println("datanumHumRefu=" + numHumanos);
}

/**
 * Actualiza los datos de la comida enviando la cantidad de comida
 * disponible.
 *
 * @param comida La cantidad de comida que se desea actualizar.
 */
public void actualizarDatosComida(int comida) {
  salida.println("datacomida=" + comida);
}

/**
 * Actualiza y envía el número total de humanos al cliente.
 *
 * @param numHumanos El número total de humanos que se desea actualizar.
```

```
 */
public void actualizarDatosHumanosTotal(int numHumanos) {
    salida.println("datanumHumTotal=" + numHumanos);
}

/**
 * Actualiza y envía el número total de zombies al cliente.
 *
 * @param numZombies El número total de zombies que se desea actualizar.
 */
public void actualizarDatosZombiesTotal(int numZombies) {
    salida.println("datanumZomTotal=" + numZombies);
}

/**
 * Actualiza los datos relacionados con un túnel específico, enviando la
 * información sobre el número de humanos presentes en dicho túnel a través
 * de la salida.
 *
 * @param tunel El identificador del túnel que se desea actualizar.
 * @param numHumanos El número de humanos presentes en el túnel
 * especificado.
 */
public void actualizarDatosTuneles(int tunel, int numHumanos) {
    salida.println("datanumHumT" + tunel + "=" + numHumanos);
}

/**
 * Actualiza la cantidad de humanos en una zona de riesgo específica y envía
 * esta información a través de la salida.
 *
 * @param zona El identificador de la zona de riesgo que se desea
 * actualizar.
 * @param numHumanos El número de humanos presentes en la zona de riesgo.
 */
public void actualizarDatosHumanosZonaRiesgo(int zona, int numHumanos) {
    salida.println("datanumHumZR" + zona + "=" + numHumanos);
}

/**
 * Actualiza los datos del número de zombis en una zona de riesgo
 * específica.
 *
 * @param zona El identificador de la zona de riesgo donde se actualizarán
```

```
     * los datos.
     * @param numZombies El número de zombis presentes en la zona de riesgo
     * especificada.
     */
    public void actualizarDatosZombiesZonaRiesgo(int zona, int numZombies) {
        salida.println("datanumZomZR" + zona + "=" + numZombies);
    }

    /**
     * Actualiza el ranking enviando los datos a través de la salida. Por cada
     * elemento en la lista de ranking, se envía una línea de texto con el
     * formato "datarankingN=valor", donde N es la posición (comenzando en 1) y
     * valor es el elemento correspondiente de la lista.
     *
     * @param ranking Una lista de cadenas que representa el ranking a
     * actualizar.
     */
    public void actualizarRanking(ArrayList<String> ranking) {
        for (int i = 0; i < ranking.size(); i++) {
            salida.println("dataranking" + (i + 1) + "=" + ranking.get(i));
        }
    }

    /**
     * Envía un comando al cliente a través del flujo de salida. Si el flujo de
     * salida no es nulo, escribe el comando en el flujo y muestra un mensaje en
     * la consola indicando que el comando fue enviado.
     *
     * @param comando El comando que se enviará al cliente.
     */
    public void enviarComando(String comando) {
        if (salida != null) {
            salida.println(comando);
            System.out.println("Comando enviado desde servidor: " + comando);
        }
    }

    /**
     * Desconecta el servidor cerrando las conexiones con el cliente.
     *
     * Este método envía un comando de salida al cliente, cierra los flujos de
     * entrada y salida, y finalmente cierra el socket del cliente. Si no hay un
     * cliente conectado, se notifica mediante un mensaje en la consola.
     *
```

```
     * @throws IOException Si ocurre un error al cerrar los flujos o el socket.
     */
    public void desconectar() throws IOException {
        if (serverSocket != null) {
            serverSocket.close();
        }
        if (clienteSocket != null) {
            enviarComando("SALIR");
            salida.close();
            entrada.close();
            clienteSocket.close();

            System.out.println("Servidor desconectado.");
        } else {
            System.out.println("No hay cliente.");
        }
    }
}


public class Ranking {
    private ArrayList<String> ranking = new ArrayList<>();
    private Servidor servidor;

    /**
     * Constructor de la clase Ranking.
     *
     * @param servidor Instancia del servidor que se utilizará para actualizar
     * el ranking.
     */
    public Ranking(Servidor servidor) {
        this.servidor = servidor;
    }

    /**
     * Actualiza el ranking con un nuevo registro de puntuación. Si el
     * identificador ya existe en el ranking, se elimina antes de agregar el
     * nuevo valor. Luego, el ranking se ordena de mayor a menor según el
     * bodycount y se limita a los tres primeros puestos. Finalmente, se
     * actualiza el ranking en el servidor y se muestra por consola.
     *
     * @param id El identificador único del jugador.
     * @param bodycount La cantidad de puntos o eliminaciones asociadas al
     * jugador.
```

```
     */
    public synchronized void actualizarRanking(String id, int bodycount) {
        // Elimina si ya está
        ranking.removeIf(entry -> entry.startsWith(id + ":"));

        // Añade el nuevo valor
        ranking.add(id + ": " + bodycount);

        // Ordena de mayor a menor
        ranking.sort((a, b) -> Integer.compare(extraerValor(b), extraerValor(a)));

        // Limita a top 3
        if (ranking.size() > 3) {
            ranking.subList(3, ranking.size()).clear();
        }

        servidor.actualizarRanking(ranking);

        // Mostrar
        System.out.println("RANKING:");
        for (String s : ranking) {
            System.out.println(" - " + s);
        }
    }

    /**
     * Extrae el valor numérico de una cadena en el formato "id: valor".
     *
     * @param entrada La cadena de entrada en el formato "id: valor".
     * @return El valor numérico extraído de la cadena.
     */
    private int extraerValor(String entrada) {
        String[] partes = entrada.split(":");
        return Integer.parseInt(partes[1].trim());
    }
}


public class EstadoPausa {
    private final ReentrantLock pauseLock = new ReentrantLock();
    private final Condition pauseCondition = pauseLock.newCondition();
    private volatile boolean paused = false;

    /**
```

```
     * Pausa la ejecución del programa estableciendo el estado de pausa en
     * verdadero. Utiliza un bloqueo para garantizar la seguridad en entornos
     * concurrentes.
     *
     * Este método asegura que el acceso al estado de pausa sea seguro al
     * utilizar un mecanismo de bloqueo (lock) para evitar condiciones de
     * carrera.
     *
     * El bloqueo se adquiere antes de modificar el estado y se libera
     * inmediatamente después, incluso si ocurre una excepción.
     */
    public void pausar() {
        pauseLock.lock();
        try {
            paused = true;
        } finally {
            pauseLock.unlock();
        }
    }

    /**
     * Reanuda la ejecución de un proceso pausado.
     *
     * Este método desbloquea el estado de pausa estableciendo la variable
     * `paused` en falso y señalando a todas las condiciones en espera que el
     * proceso puede continuar. Utiliza un bloqueo para garantizar que la
     * operación sea segura en un entorno concurrente.
     *
     * Es importante asegurarse de que este método sea llamado únicamente cuando
     * sea seguro reanudar el proceso, ya que notifica a todos los hilos en
     * espera.
     */
    public void reanudar() {
        pauseLock.lock();
        try {
            paused = false;
            pauseCondition.signalAll();
        } finally {
            pauseLock.unlock();
        }
    }

    /**
     * Método que permite a los humanos y zombies pausar su ejecución.
```

```
     *
     * Este método utiliza un bloqueo para garantizar la seguridad en entornos
     * concurrentes. Si el estado de pausa es verdadero, el hilo actual se
     * bloqueará hasta que se reanude la ejecución.
     *
     * Es importante asegurarse de que este método sea llamado únicamente cuando
     * sea seguro pausar el proceso, ya que puede bloquear el hilo actual.
     */
    public void parar() { //Método al que llaman humanos y zombies
        if (paused) {
            pauseLock.lock();
            try {
                while (paused) {
                    pauseCondition.await();
                }
            } catch (InterruptedException ex) {
                System.out.println(ex.getMessage());
            } finally {
                pauseLock.unlock();
            }
        }
    }

    /**
     * Método que verifica si el programa está en estado de pausa.
     *
     * Este método devuelve un valor booleano que indica si el estado de pausa
     * es verdadero o falso.
     *
     * @return true si el programa está en pausa, false en caso contrario.
     */
    public boolean estaPausado() {
        return paused;
    }

}



public class Log {
    private static final DateTimeFormatter formatoFecha =
DateTimeFormatter.ofPattern("yyyy-MM-dd HH:mm:ss");
    private static final String fecha_archivo =
LocalDateTime.now().format(formatoFecha).replace(":", "-").replace(" ", "_");
```

```
    private static final String carpeta = "logs/";
    private static final String nombre_archivo = carpeta + "apocalipsis" + fecha_archivo + ".txt";


    /**
     * Escribe un mensaje en un archivo de registro con un timestamp.
     *
     * Esta función es sincronizada para garantizar que múltiples hilos no escriban
     * en el archivo al mismo tiempo, evitando problemas de concurrencia.
     *
     * El archivo de registro se abre en modo de adición, por lo que los mensajes
     * nuevos se agregan al final del archivo. Cada mensaje se precede con un
     * timestamp en el formato especificado por la variable `formatoFecha`.
     *
     * En caso de que ocurra un error de entrada/salida (IOException), este se
     * captura, pero no se realiza ninguna acción adicional en el bloque catch.
     *
     * @param mensaje El mensaje que se desea escribir en el archivo de registro.
     */
    public synchronized static void escribir(String mensaje) {

        try (PrintWriter out = new PrintWriter(new FileWriter(nombre_archivo, true))) {
//try-with-resources para que Java automáticamente cierre el recurso cuando termina el bloque
try
            String timestamp = LocalDateTime.now().format(formatoFecha);
            out.println("[" + timestamp + "] " + mensaje);
        } catch (IOException e) {
        } finally {
        }
    }
}



public class Velocidad {
    private static volatile int velocidad = 1;

    /**
     * Método sincronizado para obtener el valor de la velocidad.
     *
     * @return el valor actual de la variable estática velocidad.
     */
    public static synchronized int getVelocidad() {
        return velocidad;
```

```
    }


    /**
     * Establece de manera sincronizada una nueva velocidad.
     *
     * @param nuevaVelocidad La nueva velocidad que se desea asignar.
     */
    public static synchronized void setVelocidad(int nuevaVelocidad) {
        velocidad = nuevaVelocidad;
    }

}



public class MainCliente {
    /**
     * Método principal que gestiona la conexión de un cliente a un servidor.
     *
     * Este método realiza los siguientes pasos:
     * 1. Crea una instancia de Cliente y la asocia a la pantalla principal.
     * 2. Intenta conectar al servidor mediante un diálogo modal que solicita la dirección y el
puerto.
     * 3. Permite hasta 3 intentos de conexión antes de cerrar la aplicación.
     * 4. Si la conexión es exitosa, inicia la recepción de datos y muestra la interfaz principal.
     * 5. Si no se logra conectar o el usuario decide salir, la aplicación se cierra.
     *
     * Detalles del flujo:
     * - Se utiliza un diálogo modal (JDialogCliente) para obtener los datos de conexión.
     * - Si el usuario confirma los datos, se intenta conectar al servidor.
     * - Si la conexión falla, se reduce el número de intentos restantes.
     * - Si el usuario decide salir, el programa termina.
     *
     * Notas:
     * - El método bloquea la ejecución mientras el diálogo está abierto.
     * - Si la conexión es exitosa, se inicia la recepción de datos en un hilo separado.
     *
     * @param args Argumentos de línea de comandos (no utilizados en este caso).
     */
    public static void main(String[] args) {

        Cliente cliente = new Cliente();
```

```
        ClientePantalla.getInstancia().setCliente(cliente);

        int intentos = 3;
        boolean conectado = false;
        boolean cerrar = false;
        while (intentos > 0 && !conectado && !cerrar) {
            JDialogCliente dialogo = new JDialogCliente(null, true); // modal
            dialogo.setLocationRelativeTo(null);  // centrado
            dialogo.setVisible(true);  // esto bloquea hasta que se cierre
            cerrar = dialogo.isQuiereSalir(); //si se cierra el dialogo, no se ejecuta el resto del código

            if (dialogo.isConfirmado() && !cerrar) { //si se ha confirmado el dialogo y no se ha
cerrado
                String direccion = dialogo.getDireccion();
                int puerto = dialogo.getPuerto();

                cliente.conectarAlServidor(direccion, puerto);
                if (cliente.getConectado()) {
                    System.out.println("Perfecto. Conectado");
                    conectado = true;
                } else {
                    System.out.println("No se inició la conexión. Intentos: " + (intentos - 1));
                    intentos--;
                }

            } else if (!cerrar) { //si no se ha confirmado el dialogo y no se ha cerrado
                System.out.println("No se inició la conexión. Intentos: " + (intentos - 1));
                intentos--;
            } else {
                System.out.println("Cerrando...");
            }

        }

        if (conectado) {
            cliente.iniciarRecepcion();
            ClientePantalla.getInstancia().setVisible(true);
        } else {
            System.exit(0);
        }
    }
}
```

```
public class NoSafeHaven {
    /**
     * Método principal de la aplicación NoSafeHaven.
     *
     * Este método inicializa el servidor, gestiona los intentos de conexión y,
     * si la conexión es exitosa, configura y lanza los elementos principales
     * del juego/aplicación, incluyendo zonas exteriores, refugio, ranking,
     * humanos y el paciente cero (zombie inicial).
     *
     * Flujo principal:
     * - Intenta conectar el servidor hasta un máximo de 3 intentos o hasta que
     *   el usuario decida salir.
     * - Si la conexión es exitosa, se inicializan los componentes principales
     *   del juego y se crean los hilos correspondientes para los humanos y
     *   el zombie inicial.
     * - Si la conexión falla, se termina la ejecución del programa.
     *
     * Manejo de errores:
     * - Captura excepciones generales para registrar errores en el log y
     *   mostrarlos en la consola.
     *
     * Detalles de implementación:
     * - Utiliza un diálogo modal para que el usuario configure el puerto del
     *   servidor.
     * - Implementa un sistema de pausa (EstadoPausa) para sincronizar la
     *   creación de entidades.
     * - Los humanos y el zombie inicial se ejecutan en hilos separados.
     *
     * Notas:
     * - El número de humanos creados puede ser configurado modificando la
     *   variable `numHumanos`.
     * - La velocidad de creación de humanos está influenciada por la clase
     *   `Velocidad`.
     *
     * @param args Argumentos de línea de comandos (no utilizados en este caso).
     */
    public static void main(String[] args) {
        try {
            Servidor servidor = new Servidor();
            PantallaPrincipal.getInstancia().setServidor(servidor);

            int intentos = 3;
            boolean conectado = false;
```

```
        boolean cerrar = false;
        while (intentos > 0 && !conectado && !cerrar) {
            JDialogServidor dialogo = new JDialogServidor(null, true); // modal
            dialogo.setLocationRelativeTo(null);  // centrado
            dialogo.setVisible(true);  // esto bloquea hasta que se cierre
            cerrar = dialogo.isQuiereSalir(); //si se cierra el dialogo, no se ejecuta el resto del
código

            if (dialogo.isConfirmado() && !cerrar) { //si se ha confirmado el dialogo y no se ha
cerrado
                int puerto = dialogo.getPuerto();
                servidor.iniciarServidor(puerto);
                if (servidor.getConectado()) {
                    System.out.println("Perfecto. Conectado");
                    conectado = true;
                } else {
                    System.out.println("No se inició la conexión. Intentos: " + (intentos - 1));
                    intentos--;
                }

            } else if (!cerrar) { //si no se ha confirmado el dialogo y no se ha cerrado
                System.out.println("No se inició la conexión. Intentos: " + (intentos - 1));
                intentos--;
            }
        }

        if (conectado) { //si no está conectado por que hay 2 o más servidores a la vez y no
pueden usar el mismo puerto, no ejecuta el resto del código
            PantallaPrincipal.getInstancia().setVisible(true);

            EstadoPausa ep = new EstadoPausa();
            servidor.setEstadopausa(ep);

            Refugio refugio = new Refugio(servidor);
            servidor.setRefugio(refugio); // Asignar el refugio al servidor, para que pueda acceder
a él en las conexiones pertinentes

            ZonaExterior[] zonas = new ZonaExterior[4];
            Ranking r = new Ranking(servidor);

            for (int i = 0; i < 4; i++) {
                zonas[i] = new ZonaExterior(i, servidor);
            }
```

```
            ep.parar();
            Zombie zombie = new Zombie("Z00000", zonas, r, ep); // Crear paciente 0
            zombie.start(); // Arrancar el hilo del Zombie
            Log.escribir("Creando zombie Z00000");
            System.out.println("Creando zombie Z00000");

            ep.parar();
            int numHumanos = 10000; // Puedes cambiar el número que quieras
            for (int i = 1; i <= numHumanos; i++) {  // Empezar en 1 para que el id sea H00001, y
no H00000 para no tener dos zombis con id Z00000
                String id = "H" + String.format("%05d", i); // Formatear el número con ceros a la
izquierda
                Humano humano = new Humano(id, refugio, zonas, r, ep);
                humano.start(); // Arrancar cada hilo de Humano
                Log.escribir("Creando humano " + id);
                System.out.println("Creando humano " + id);
                ep.parar();
                try {
                    Thread.sleep((int) (Math.random() * 500 + 1500) / Velocidad.getVelocidad());
                } catch (Exception e) {
                    System.out.println(e.getMessage());
                }

            }
        } else {
            System.out.println("no se pudo crear el servidor");
            System.exit(0);
        }
    } catch (Exception e) {
        Log.escribir("Error en el main: " + e.getMessage());
        System.out.println("Error en el main " + e.getMessage());
    }
  }
}



public final class ClientePantalla extends javax.swing.JFrame {
   // Instancia estática y volatile para evitar problemas de visibilidad entre hilos
   private static volatile ClientePantalla instancia;

   private static volatile boolean paused = false;

   private Cliente cliente;
```

```
/**
 * Constructor privado para evitar la creación de instancias
 */
private ClientePantalla() {
    initComponents();
}

/**
 * Establece el cliente asociado a esta pantalla.
 *
 * @param cliente El objeto Cliente que se desea asociar.
 */
public void setCliente(Cliente cliente) {
    this.cliente = cliente;
}

/**
 * Obtiene la instancia única de la clase ClientePantalla utilizando el
 * patrón Singleton. Este método garantiza que solo se cree una única
 * instancia de ClientePantalla, incluso en un entorno multihilo, mediante
 * el uso de sincronización y doble comprobación.
 *
 * @return La instancia única de ClientePantalla.
 */
public static ClientePantalla getInstancia() {
    // Primero se verifica si la instancia ya está creada
    ClientePantalla result = instancia;
    if (result != null) {
        return result; // Si la instancia ya está creada, se retorna
    }

    // Si no, se sincroniza el bloque para garantizar que solo un hilo
    // pueda crear la instancia a la vez
    synchronized (ClientePantalla.class) {
        if (instancia == null) {
            // Doble comprobación: si la instancia aún es null, se crea
            instancia = new ClientePantalla();
        }
        return instancia;
    }
}

/**
```

```
     * Actualiza el texto de la etiqueta de comida en la interfaz gráfica. Este
     * método asegura que la actualización se realice en el hilo de despacho de
     * eventos de Swing para evitar problemas de concurrencia.
     *
     * @param comida La cantidad o descripción de comida que se mostrará en la
     * etiqueta.
     */
    public void actualizarComida(String comida) {
        SwingUtilities.invokeLater(() -> {
            this.comida.setText("Comida: " + comida);
        });
    }

    /**
     * Actualiza la información del refugio en la interfaz gráfica de usuario.
     *
     * La actualización se realiza en el hilo de la interfaz gráfica (Event
     * Dispatch Thread) utilizando SwingUtilities.invokeLater para garantizar la
     * seguridad en el manejo de la GUI.
     *
     * @param num El número de humanos en el refugio que se mostrará en la
     * etiqueta.
     */
    public void actualizarRefugio(String num) {
        SwingUtilities.invokeLater(() -> {
            this.NumHumanos.setText("Nº humanos: " + num);
        });
    }

    /**
     * Actualiza el número de humanos en la interfaz gráfica de usuario.
     *
     * La actualización se realiza en el hilo de la interfaz gráfica (Event
     * Dispatch Thread) utilizando SwingUtilities.invokeLater para garantizar la
     * seguridad en el manejo de la GUI.
     *
     * @param num El número de humanos que se mostrará en la etiqueta.
     */
    public void actualizarTotalHumanos(String num) {
        SwingUtilities.invokeLater(() -> {
            this.humanos.setText(num);
        });
    }
```

```
/**
 * Actualiza el número de zombies en la interfaz gráfica de usuario.
 *
 * La actualización se realiza en el hilo de la interfaz gráfica (Event
 * Dispatch Thread) utilizando SwingUtilities.invokeLater para garantizar la
 * seguridad en el manejo de la GUI.
 *
 * @param num El número de zombies que se mostrará en la etiqueta.
 */
public void actualizarTotalZombies(String num) {
    SwingUtilities.invokeLater(() -> {
        this.zombies.setText(num);
    });
}

/**
 * Actualiza el número de humanos en un túnel específico.
 *
 * La actualización se realiza en el hilo de la interfaz gráfica (Event
 * Dispatch Thread) utilizando SwingUtilities.invokeLater para garantizar la
 * seguridad en el manejo de la GUI.
 *
 * @param tunel El número del túnel que se actualizará (0-3).
 * @param num El número de humanos que se mostrará en la etiqueta del túnel.
 */
public void actualizarTunel(int tunel, String num) {
    SwingUtilities.invokeLater(() -> {
        switch (tunel) {
            case 0:
                T1.setText("Nº humanos: " + num);
                break;
            case 1:
                T2.setText("Nº humanos: " + num);
                break;
            case 2:
                T3.setText("Nº humanos: " + num);
                break;
            case 3:
                T4.setText("Nº humanos: " + num);
                break;
        }
    });
}
```

```
/**
 * Actualiza la información del número de humanos en un exterior específico
 * de la interfaz gráfica.
 *
 * La actualización de la interfaz gráfica se realiza en el hilo de eventos
 * de Swing utilizando {@code SwingUtilities.invokeLater} para garantizar la
 * seguridad en el acceso a los componentes de la interfaz.
 *
 * @param exterior El número del exterior que se actualizará (0-3).
 * @param numH El número de humanos que se mostrará en la etiqueta del
 * exterior.
 */
public void actualizarExteriorHumanos(int exterior, String numH) {
    SwingUtilities.invokeLater(() -> {
        switch (exterior) {
            case 0:
                NH1.setText("Nº humanos: " + numH);
                break;
            case 1:
                NH2.setText("Nº humanos: " + numH);
                break;
            case 2:
                NH3.setText("Nº humanos: " + numH);
                break;
            case 3:
                NH4.setText("Nº humanos: " + numH);
                break;
        }
    });
}

/**
 * Actualiza la información del número de zombies en un exterior específico
 * de la interfaz gráfica.
 *
 * La actualización de la interfaz gráfica se realiza en el hilo de eventos
 * de Swing utilizando {@code SwingUtilities.invokeLater} para garantizar la
 * seguridad en el acceso a los componentes de la interfaz.
 *
 * @param exterior El número del exterior que se actualizará (0-3).
 * @param numZ El número de zombies que se mostrará en la etiqueta del
 * exterior.
 */
public void actualizarExteriorZombies(int exterior, String numZ) {
```

```
    SwingUtilities.invokeLater(() -> {
        switch (exterior) {
            case 0:
                NZ1.setText("Nº Zombies: " + numZ);
                break;
            case 1:
                NZ2.setText("Nº Zombies: " + numZ);
                break;
            case 2:
                NZ3.setText("Nº Zombies: " + numZ);
                break;
            case 3:
                NZ4.setText("Nº Zombies: " + numZ);
                break;
        }
    });
}

/**
 * Actualiza el texto de un componente de la interfaz gráfica basado en el
 * ranking proporcionado.
 *
 * La actualización del texto se realiza en el hilo de la interfaz gráfica
 * (Event Dispatch Thread) utilizando SwingUtilities.invokeLater para
 * garantizar la seguridad en el manejo de la interfaz.
 *
 * @param ranking El número del ranking (1, 2 o 3) que determina qué
 * componente actualizar. Debe ser un valor entero entre 1 y 3.
 * @param valor El texto que se establecerá en el componente correspondiente
 * al ranking.
 */
public void actualizarRanking(int ranking, String valor) {
    SwingUtilities.invokeLater(() -> {
        switch (ranking) {
            case 1:
                z1.setText(valor);
                break;
            case 2:
                z2.setText(valor);
                break;
            case 3:
                z3.setText(valor);
                break;
        }
```

```
        });
    }

(...) //Cosas generadas automáticamente de la interfaz
private void stopButtonActionPerformed(java.awt.event.ActionEvent evt) {
        paused = !paused;  // Alterna estado de pausa/reanudación
        if (!paused) {
            cliente.enviarComando("REANUDAR");
            stopButton.setBackground(Color.red);
            System.out.println("reanudar");
        } else {
            cliente.enviarComando("PAUSAR");
            stopButton.setBackground(Color.green);
            System.out.println("parandoo");
        }
    }

private void formWindowClosing(java.awt.event.WindowEvent evt) {
        if (cliente != null) {
            try {
                cliente.desconectar();
            } catch (IOException ex) {
                System.err.println("Error al desconectar cliente: " + ex.getMessage());
            }
        }
        System.exit(0);
    }

    private void velocidad1ActionPerformed(java.awt.event.ActionEvent evt) {
        velocidad1.setBackground(Color.GREEN);
        velocidad5.setBackground(Color.WHITE);
        velocidad10.setBackground(Color.WHITE);
        System.out.println("velocidad=1");
        cliente.enviarComando("VELOCIDAD=1");
    }

    private void velocidad5ActionPerformed(java.awt.event.ActionEvent evt) {
        velocidad1.setBackground(Color.WHITE);
        velocidad5.setBackground(Color.GREEN);
        velocidad10.setBackground(Color.WHITE);
        System.out.println("velocidad=5");
        cliente.enviarComando("VELOCIDAD=5");
    }
```

```
    private void velocidad10ActionPerformed(java.awt.event.ActionEvent evt) {
        velocidad1.setBackground(Color.WHITE);
        velocidad5.setBackground(Color.WHITE);
        velocidad10.setBackground(Color.GREEN);
        System.out.println("velocidad=10");
        cliente.enviarComando("VELOCIDAD=10");
    }

 private void BotonComidaExtraActionPerformed(java.awt.event.ActionEvent evt) {
        cliente.enviarComando("COMIDA_EXTRA=50");
    }

(...)

 public static void main(String args[]) {
(...)
java.awt.EventQueue.invokeLater(new Runnable() {
            public void run() {
                ClientePantalla.getInstancia().setVisible(true);
            }
        });
    }

(....)
}
```

```
public final class PantallaPrincipal extends javax.swing.JFrame {
    private Servidor servidor; //para desconectar al pulsar cerrar
    private Set<String> heridos = new HashSet(); // Conjunto para almacenar los IDs de los
heridos
    private static final ReentrantLock heridosLock = new ReentrantLock();

    SimpleAttributeSet estiloNormal = new SimpleAttributeSet();
    SimpleAttributeSet estiloHerido = new SimpleAttributeSet();

    // Instancia estática y volatile para evitar problemas de visibilidad entre hilos
    private static volatile PantallaPrincipal instancia;

    /**
     * Constructor privado de la clase PantallaPrincipal. Inicializa los
     * componentes de la interfaz gráfica de usuario. Este constructor está
     * diseñado para ser utilizado internamente y no permite la creación directa
```

```
     * de instancias desde fuera de la clase.
     */
    private PantallaPrincipal() {
        initComponents();
    }

    /**
     * Establece el servidor que será utilizado por la aplicación.
     *
     * @param servidor El objeto de tipo Servidor que se asignará a esta
     * instancia.
     */
    public void setServidor(Servidor servidor) {
        this.servidor = servidor;
    }

    /**
     * Método estático para obtener la instancia única de la clase
     * PantallaPrincipal. Implementa el patrón Singleton con doble verificación
     * para garantizar que solo se cree una instancia de la clase, incluso en un
     * entorno multihilo.
     *
     * @return La instancia única de PantallaPrincipal.
     */
    public static PantallaPrincipal getInstancia() {
        // Primero se verifica si la instancia ya está creada
        PantallaPrincipal result = instancia;
        if (result != null) {
            return result; // Si la instancia ya está creada, se retorna
        }

        // Si no, se sincroniza el bloque para garantizar que solo un hilo
        // pueda crear la instancia a la vez
        synchronized (PantallaPrincipal.class) {
            if (instancia == null) {
                // Doble comprobación: si la instancia aún es null, se crea
                instancia = new PantallaPrincipal();
            }
            return instancia;
        }
    }

    /**
     * Actualiza la etiqueta de comida en la interfaz gráfica con el valor
```

```
     * proporcionado. Este método asegura que la actualización se realice en el
     * hilo de eventos de Swing.
     *
     * @param comida El nuevo valor de comida que se mostrará en la etiqueta.
     */
    public void actualizarComida(int comida) {
        SwingUtilities.invokeLater(() -> {
            this.comida.setText("Comida: " + comida);
        });
    }

    /**
     * Actualiza el contador de humanos en la interfaz gráfica con el valor
     * proporcionado. Este método asegura que la actualización se realice en el
     * hilo de eventos de Swing.
     *
     * @param humanos El nuevo valor de humanos que se mostrará en el contador.
     */
    public void actualizarHumanos(int humanos) {
        SwingUtilities.invokeLater(() -> {
            this.counterHumanos.setText("Humanos: " + humanos);
        });
    }

    /**
     * Actualiza el contenido de la zona común en la interfaz gráfica con una
     * lista de IDs.
     *
     * La actualización de la interfaz gráfica se realiza en el hilo de eventos
     * de Swing utilizando SwingUtilities.invokeLater para garantizar la
     * seguridad en el acceso a componentes de la interfaz.
     *
     * @param listaIDs La lista de IDs que se mostrará en la zona común. Se
     * espera que la lista contenga los IDs de los humanos presentes en la zona
     * común. La lista se mostrará en la interfaz gráfica, separando los IDs por
     * espacios y agregando un salto de línea cada 4 IDs.
     */
    public void actualizarZonaComun(CopyOnWriteArrayList<String> listaIDs) {
        StringBuilder sb = new StringBuilder();
        int count = 0;
        for (String id : listaIDs) {
            sb.append(id);
            count++;
            if (count % 4 == 0) {
```

```
            sb.append("\n");
        } else {
            sb.append(" ");
        }
    }
    SwingUtilities.invokeLater(() -> {
        ZonaComun.setText(sb.toString());
    });
}

/**
 * Actualiza el contenido del componente "Comedor" con una lista de IDs
 * proporcionada.
 *
 * @param listaIDs Una lista segura para subprocesos (CopyOnWriteArrayList)
 * que contiene los IDs que se mostrarán en el componente "Comedor". Cada ID
 * se mostrará en una línea separada.
 */
public void actualizarComedor(CopyOnWriteArrayList<String> listaIDs) {
    actualizarTextPaneConColor(Comedor, listaIDs, heridos, 4); // 1 ID por línea
}

/**
 * Actualiza el contenido del componente "Zona de Descanso" con una lista de
 * IDs proporcionada.
 *
 * @param listaIDs Una lista segura para subprocesos (CopyOnWriteArrayList)
 * que contiene los IDs que se mostrarán en el componente "Zona de
 * Descanso". Cada ID se mostrará en una línea separada.
 */
public void actualizarZonaDescanso(CopyOnWriteArrayList<String> listaIDs) {
    actualizarTextPaneConColor(ZonaDescanso, listaIDs, heridos, 4); // 4 IDs por línea
}

/**
 * Actualiza el contenido de un túnel específico con una lista de IDs
 * proporcionada. Dependiendo del túnel especificado, se actualiza el
 * componente gráfico correspondiente.
 *
 * @param tunel El número del túnel a actualizar (0, 1, 2 o 3).
 * @param listaIDs Una lista de IDs que se utilizarán para actualizar el
 * contenido del túnel.
 */
public void actualizarTunel(int tunel, CopyOnWriteArrayList<String> listaIDs) {
```

```
    switch (tunel) {
        case 0:
            actualizarTextPaneConColor(DentroT1, listaIDs, heridos, 1);
            break;
        case 1:
            actualizarTextPaneConColor(DentroT2, listaIDs, heridos, 1);
            break;
        case 2:
            actualizarTextPaneConColor(DentroT3, listaIDs, heridos, 1);
            break;
        case 3:
            actualizarTextPaneConColor(DentroT4, listaIDs, heridos, 1);
            break;
    }
}

/**
 * Actualiza el contenido del túnel medio con una lista de IDs
 * proporcionada. Dependiendo del túnel especificado, se actualiza el
 * componente gráfico correspondiente.
 *
 * @param tunel El número del túnel a actualizar (0, 1, 2 o 3).
 * @param listaIDs Una lista de IDs que se utilizarán para actualizar el
 * contenido del túnel medio.
 */
public void actualizarTunelMedio(int tunel, CopyOnWriteArrayList<String> listaIDs) {
    switch (tunel) {
        case 0:
            actualizarTextPaneConColor(MedioT1, listaIDs, heridos, 1);
            break;
        case 1:
            actualizarTextPaneConColor(MedioT2, listaIDs, heridos, 1);
            break;
        case 2:
            actualizarTextPaneConColor(MedioT3, listaIDs, heridos, 1);
            break;
        case 3:
            actualizarTextPaneConColor(MedioT4, listaIDs, heridos, 1);
            break;
    }
}

/**
 * Actualiza el contenido del túnel fuera con una lista de IDs
```

```
     * proporcionada. Dependiendo del túnel especificado, se actualiza el
     * componente gráfico correspondiente.
     *
     * @param tunel El número del túnel a actualizar (0, 1, 2 o 3).
     * @param listaIDs Una lista de IDs que se utilizarán para actualizar el
     * contenido del túnel fuera.
     */
    public void actualizarTunelFuera(int tunel, CopyOnWriteArrayList<String> listaIDs) {
        switch (tunel) {
            case 0:
                actualizarTextPaneConColor(FueraT1, listaIDs, heridos, 1);
                break;
            case 1:
                actualizarTextPaneConColor(FueraT2, listaIDs, heridos, 1);
                break;
            case 2:
                actualizarTextPaneConColor(FueraT3, listaIDs, heridos, 1);
                break;
            case 3:
                actualizarTextPaneConColor(FueraT4, listaIDs, heridos, 1);
                break;
        }
    }

    /**
     * Actualiza el contenido de los exteriores con una lista de IDs
     * proporcionada. Dependiendo del exterior especificado, se actualiza el
     * componente gráfico correspondiente.
     *
     * @param exterior El número del exterior a actualizar (0, 1, 2 o 3).
     * @param listaIDs Una lista de IDs que se utilizarán para actualizar el
     * contenido del exterior.
     */
    public void actualizarExteriorHumanos(int exterior, CopyOnWriteArrayList<String> listaIDs) {
        StringBuilder sb = new StringBuilder();
        int count = 0;
        for (String id : listaIDs) {
            sb.append(id);
            count++;
            if (count % 3 == 0) {
                sb.append("\n");
            } else {
                sb.append(" ");
            }
```

```
    }
    SwingUtilities.invokeLater(() -> {
        switch (exterior) {
            case 0:
                Exterior1.setText(sb.toString());
                break;
            case 1:
                Exterior2.setText(sb.toString());
                break;
            case 2:
                Exterior3.setText(sb.toString());
                break;
            case 3:
                Exterior4.setText(sb.toString());
                break;
        }
    });
}

/**
 * Actualiza el contenido de los exteriores con una lista de IDs de zombis
 * proporcionada. Dependiendo del exterior especificado, se actualiza el
 * componente gráfico correspondiente.
 *
 * @param exterior El número del exterior a actualizar (0, 1, 2 o 3).
 * @param listaIDs Una lista de IDs que se utilizarán para actualizar el
 * contenido del exterior.
 */
public void actualizarExteriorZombies(int exterior, CopyOnWriteArrayList<String> listaIDs) {
    StringBuilder sb = new StringBuilder();
    int count = 0;
    for (String id : listaIDs) {
        sb.append(id);
        count++;
        if (count % 3 == 0) {
            sb.append("\n");
        } else {
            sb.append(" ");
        }
    }
    SwingUtilities.invokeLater(() -> {
        switch (exterior) {
            case 0:
                ExteriorZ1.setText(sb.toString());
```

```
                break;
            case 1:
                ExteriorZ2.setText(sb.toString());
                break;
            case 2:
                ExteriorZ3.setText(sb.toString());
                break;
            case 3:
                ExteriorZ4.setText(sb.toString());
                break;
        }
    });
}

/**
 * Agrega un identificador de herido a la lista de heridos de manera segura
 * utilizando un bloqueo para garantizar la sincronización en un entorno
 * concurrente.
 *
 * @param id El identificador único del herido que se desea agregar a la
 * lista.
 */
public void addHerido(String id) {
    heridosLock.lock();
    try {
        heridos.add(id);
    } finally {
        heridosLock.unlock();
    }
}

/**
 * Elimina un identificador de herido de la lista de heridos de manera
 * segura utilizando un bloqueo para garantizar la sincronización en un
 * entorno concurrente.
 *
 * @param id El identificador único del herido que se desea eliminar de la
 * lista.
 */
public void quitarHerido(String id) {
    heridosLock.lock();
    try {
        heridos.remove(id);
    } finally {
```

```
            heridosLock.unlock();
        }
    }

    /**
     * Actualiza el contenido de un JTextPane con una lista de IDs, aplicando
     * colores específicos según si los IDs están en un conjunto de IDs heridos.
     * Además, organiza los IDs en líneas según el número especificado por
     * parámetro.
     *
     * @param textPane El JTextPane que se actualizará con el contenido.
     * @param listaIDs Una lista de IDs que se mostrarán en el JTextPane.
     * @param idsHeridos Un conjunto de IDs que serán resaltados en color rojo.
     * @param porLinea Número de IDs que se mostrarán por línea. Si es 1, los
     * IDs se separarán por espacios; si es mayor que 1, se insertará un salto
     * de línea después de cada grupo de IDs.
     */
    private void actualizarTextPaneConColor(JTextPane textPane,
CopyOnWriteArrayList<String> listaIDs, Set<String> idsHeridos, int porLinea) {
        // Crear contenido completo fuera del EDT
        javax.swing.text.StyledDocument nuevoDoc = new
javax.swing.text.DefaultStyledDocument();

        StyleConstants.setForeground(estiloNormal, Color.BLACK);
        StyleConstants.setForeground(estiloHerido, Color.RED);

        int count = 0;
        for (String id : listaIDs) {
            SimpleAttributeSet estilo = idsHeridos.contains(id) ? estiloHerido : estiloNormal;
            try {
                nuevoDoc.insertString(nuevoDoc.getLength(), id, estilo);
                count++;
                if (porLinea == 1 && count % porLinea == 0) {
                    nuevoDoc.insertString(nuevoDoc.getLength(), " ", estiloNormal);
                } else if (porLinea > 0 && count % porLinea == 0) {
                    nuevoDoc.insertString(nuevoDoc.getLength(), "\n", estiloNormal);
                } else {
                    nuevoDoc.insertString(nuevoDoc.getLength(), " ", estiloNormal);
                }
            } catch (BadLocationException e) {
                 System.out.println(e.getMessage());
            }
        }
```

```
        // Reemplazar el documento completo en el EDT
        SwingUtilities.invokeLater(() -> textPane.setDocument(nuevoDoc));
    }

(...)

private void formWindowClosing(java.awt.event.WindowEvent evt) {
        if (servidor != null) {
            try {
                servidor.desconectar();
            } catch (IOException ex) {
                System.err.println("Error al desconectar servidor: " + ex.getMessage());
            }
        }
        System.exit(0);
    }

public static void main(String args[]) {
(...)
        java.awt.EventQueue.invokeLater(new Runnable() {
            public void run() {
                PantallaPrincipal.getInstancia().setVisible(true);
            }
        });
    }


public class JDialogCliente extends javax.swing.JDialog {
    private String direccion;
    private int puerto;
    private boolean confirmado = false;
    private boolean quiereSalir = false; //para cerrar el dialogo sin confirmar

    /**
     * Creates new form JDialogCliente
     */
    public JDialogCliente(java.awt.Frame parent, boolean modal) {
        super(parent, modal);
        initComponents();
    }

    public String getDireccion() {
        return direccion;
    }
```

```
    public int getPuerto() {
        return puerto;
    }

    public boolean isConfirmado() {
        return confirmado;
    }

    public boolean isQuiereSalir() {
        return quiereSalir;
    }

(...)

private void puertoTextFieldMouseClicked(java.awt.event.MouseEvent evt) {
        puertoTextField.setText("");
    }

/**
     * Maneja el evento de acción del botón "Listo" para establecer la conexión.
     *
     * Este método se ejecuta cuando el usuario hace clic en el botón "Listo" en
     * la interfaz. Valida la dirección y el puerto ingresados por el usuario, y
     * si son válidos, los asigna a las variables correspondientes y cierra el
     * diálogo.
     *
     * Validaciones realizadas: - La dirección no debe estar vacía. - El puerto
     * debe estar en el rango de 1024 a 65535.
     *
     * En caso de que el puerto no sea un número válido, se muestra un mensaje
     * de error. Si ocurre cualquier otro error, se muestra un mensaje con la
     * descripción del problema.
     *
     * @param evt El evento de acción generado al hacer clic en el botón.
     */
    private void botonListoConexionActionPerformed(java.awt.event.ActionEvent evt) {

        try {
            String dir = direccionTextField.getText();
            int port = Integer.parseInt(puertoTextField.getText());

            if (dir.isEmpty()) {
                JOptionPane.showMessageDialog(this, "La dirección no puede estar vacía");
```

```
            return;
        }

        if (port < 1024 || port > 65535) {
            JOptionPane.showMessageDialog(this, "Puerto fuera de rango (1024–65535)");
            return;
        }

        direccion = dir;
        puerto = port;
        confirmado = true;

        dispose();  // Cierra el JDialog

    } catch (NumberFormatException ex) {
        JOptionPane.showMessageDialog(this, "Puerto inválido");
    } catch (Exception ex) {
        JOptionPane.showMessageDialog(this, "Error al iniciar el servidor: " +
ex.getMessage());
    }
  }

private void direccionTextFieldMouseClicked(java.awt.event.MouseEvent evt) {
    direccionTextField.setText("");
  }

private void formWindowClosing(java.awt.event.WindowEvent evt) {
    quiereSalir = true; //marcamos que quiere salir sin confirmar

  }

 public static void main(String args[]) {
(...)
java.awt.EventQueue.invokeLater(new Runnable() {
        public void run() {
            JDialogCliente dialog = new JDialogCliente(new javax.swing.JFrame(), true);
            dialog.addWindowListener(new java.awt.event.WindowAdapter() {
                @Override
                public void windowClosing(java.awt.event.WindowEvent e) {
                    System.exit(0);
                }
            });
            dialog.setVisible(true);
        }
```

```
        });
    }

public class JDialogServidor extends javax.swing.JDialog {
    private int puerto;
    private boolean confirmado = false;
    private boolean quiereSalir = false;

public JDialogServidor(java.awt.Frame parent, boolean modal) {
        super(parent, modal);
        initComponents();
    }

    public int getPuerto() {
        return puerto;
    }

    public boolean isConfirmado() {
        return confirmado;
    }

    public boolean isQuiereSalir() {
        return quiereSalir;
    }

 private void puertoTextFieldMouseClicked(java.awt.event.MouseEvent evt) {
        puertoTextField.setText("");
    }

/**
     * Maneja el evento de acción del botón "Listo" para establecer la conexión.
     *
     * Este método se ejecuta cuando el usuario hace clic en el botón "Listo" en
     * la interfaz gráfica. Valida el puerto ingresado por el usuario,
     * asegurándose de que esté dentro del rango permitido (1024–65535). Si el
     * puerto es válido, lo asigna a la variable correspondiente y marca la
     * conexión como confirmada. En caso de error, muestra un mensaje de
     * advertencia o error al usuario.
     *
     * @param evt El evento de acción generado por el clic del usuario en el
     * botón.
     *
     * Excepciones manejadas: - NumberFormatException: Si el texto ingresado no
     * es un número válido. - Exception: Cualquier otro error inesperado durante
```

```
     * la validación o el procesamiento.
     */
    private void botonListoConexionActionPerformed(java.awt.event.ActionEvent evt) {
        try {
            int port = Integer.parseInt(puertoTextField.getText());

            if (port < 1024 || port > 65535) {
                JOptionPane.showMessageDialog(this, "Puerto fuera de rango (1024–65535)");
                return;
            }
            puerto = port;
            confirmado = true;

            dispose();  // Cierra el JDialog

        } catch (NumberFormatException ex) {
            JOptionPane.showMessageDialog(this, "Puerto inválido");
        } catch (Exception ex) {
            JOptionPane.showMessageDialog(this, "Error al iniciar el servidor: " +
ex.getMessage());
        }
    }

private void formWindowClosing(java.awt.event.WindowEvent evt) {
        quiereSalir = true;
    }

 public static void main(String args[]) {
(...)
java.awt.EventQueue.invokeLater(new Runnable() {
            public void run() {
                JDialogServidor dialog = new JDialogServidor(new javax.swing.JFrame(), true);
                dialog.addWindowListener(new java.awt.event.WindowAdapter() {
                    @Override
                    public void windowClosing(java.awt.event.WindowEvent e) {
                        System.exit(0);
                    }
                });
                dialog.setVisible(true);
            }
        });
    }
```